

SPEC CPU2026: Characterization, Representativeness, and Cross-Suite Comparison

Ruihao Li Andrew Jacob Neeraja J. Yadwadkar Lizy K. John
The University of Texas at Austin

Abstract

Specialized accelerators dominate AI workloads, but CPUs remain critical for orchestrating these accelerators and running other daily services. As a result, CPU performance increasingly shapes end-to-end system efficiency, making it necessary for benchmarks to reflect modern workloads and bottlenecks. Yet it remains unclear whether the newest general-purpose CPU benchmark suite changes the architectural conclusions researchers would draw from prior *SPEC CPU* generations. We present the first comprehensive characterization of *SPEC CPU2026* across nine recent Intel, AMD, Ampere, and Nvidia platforms. Compared with *SPEC CPU2017*, *SPEC CPU2026* is not merely a successor: it increases instruction volume and memory footprint, and shifts pressure toward emerging bottlenecks, most notably higher instruction-cache stress.

These shifts raise two practical questions: how much of the new suite is needed to preserve its behavioral coverage, and how should its coverage be interpreted relative to modern domain-specific suites? Using clustering-based representativeness analysis, we identify compact subsets of 4-5 workloads per group that preserve 96.4-99.9% of full-suite behavior, reducing evaluation cost while maintaining fidelity. We then compare *SPEC CPU2026* against *SPEC CPU2017*, DCPerf, and MLPerf using cross-suite microarchitectural metrics. *SPEC CPU2026* remains a complementary general-purpose suite: it moves closer to datacenter-like frontend pressure than prior *SPEC CPU* generations, while remaining less vector-intensive than MLPerf and less frontend-extreme than DCPerf. Finally, we show that *SPEC CPU2026* supports architectural studies beyond aggregate scores through case studies on page sizes and allocators, prefetching, compiler optimizations, ISA sensitivity, many-core scaling, and rolling round-robin based heterogeneous proxy workloads for selected DCPerf-like pressure points. Overall, *SPEC CPU2026* updates the standardized general-purpose CPU baseline for the next decade of architecture evaluation.

1 Introduction

To keep pace with the rapid scaling of machine learning (ML) models [79, 106, 107, 131], the architecture community has increasingly pivoted toward specialized compute paradigms. Recent systems leverage GPUs [3, 26, 67, 94, 105, 112, 147], FPGAs [2, 22, 35, 114, 144], and custom domain-specific accelerators [24, 27, 34, 51, 52, 58, 59, 139] to sustain next-generation ML performance. This trend has fostered the perception that CPU-oriented benchmarking is a “solved” problem, or at least less relevant in an accelerator-dominated era. In practice, however, general-purpose CPUs remain indispensable: they run desktop and datacenter services [40, 62, 85, 123, 148], coordinate heterogeneous accelerators [57, 82, 132, 142, 150], execute latency-sensitive control and systems code [6, 93, 140], and provide a fallback for workloads that do not map efficiently to specialized hardware [55, 83, 137, 143]. Recent industry reports

also point to renewed CPU demand associated with Agentic AI deployment [8, 60, 104]. Considering all these trends, rigorous CPU evaluation will not become less – but even more – important and necessary in computer architecture research.

SPEC CPU has long been the de facto standard for CPU performance evaluation in both industry and academia [48, 49, 97, 98, 121, 141]. The release of *SPEC CPU2017* (*SPEC CPU17*) prompted extensive characterization studies [18, 47, 72, 87, 91, 110, 116, 122], extending the community’s understanding beyond *SPEC CPU2006* [49, 98, 121, 141]. Since then, ML and cloud/datacenter workloads have become mainstream, and suites such as MLPerf [106] and DCPerf [128] have emerged to capture these domains. This raises a timely question: what does the newest *SPEC CPU* release – *SPEC CPU2026* (*SPEC CPU26*) – contribute beyond *SPEC CPU17*, and how should it be interpreted relative to modern domain-focused suites?

In this paper, we present the first comprehensive characterization of *SPEC CPU26* and its implications for modern CPU architecture evaluation. Using microarchitectural characterization and cross-generation similarity analysis, we show that *SPEC CPU26* updates the general-purpose CPU evaluation baseline: it provides meaningful new stress coverage over *SPEC CPU17* while remaining complementary to DCPerf and MLPerf. Specifically, it shifts stress toward larger instruction footprints, larger working sets, broader compiler and ISA sensitivity, and higher many-core scalability, while reducing dependence on several extreme *SPEC CPU17* outliers. These changes matter because 2026-era CPU research increasingly studies high-core-count scaling [125, 127], chiplet and non-chiplet organization [1, 64, 126], heterogeneous memory systems [120, 129], cloud-style co-location [23, 76], and compiler/runtime configuration [14, 146] rather than only single-core peak performance.

To understand *SPEC CPU26* better, we answer four questions that matter to the computer architecture community:

- How does *SPEC CPU26* differ from *SPEC CPU17* in microarchitectural behavior and coverage?
- Can compact subsets preserve *SPEC CPU26*’s behavioral diversity at lower evaluation cost?
- How does *SPEC CPU26* compare with DCPerf and MLPerf?
- How can *SPEC CPU26* support practical evaluation beyond score reporting?

In summary, this paper makes the following contributions:

- We provide the first comprehensive characterization of *SPEC CPU26* across nine recent platforms, showing how it differs from *SPEC CPU17* in instruction volume, memory footprint, frontend pressure, branch behavior, and scaling behavior (§ 2).
- We perform redundancy analysis and representative subset selection (§ 3), showing that 4-5 workloads per group preserve 96.4-99.9% of suite behavior and substantially reduce evaluation time with minimal coverage loss.

- We compare *SPEC CPU26* with *SPEC CPU17*, DCPerf, and MLPerf (§ 4), finding that *SPEC CPU26* moves the general-purpose CPU baseline closer to datacenter-like frontend pressure while remaining distinct from the most extreme service and ML inference behaviors.
- We demonstrate practical use cases of *SPEC CPU26* for 2026-era CPU research (§ 5): compared with *SPEC CPU17*, *SPEC CPU26* is more sensitive to compiler optimizations, shows broader ISA-dependent variation, sustains higher many-core scaling, and enables heterogeneous proxy workload generation via stagger mode.

Overall, *SPEC CPU26* is a timely benchmark suite that updates standardized, comparable CPU evaluation for the coming decade.

2 Overview of *SPEC CPU26*

This section summarizes *SPEC CPU26*'s organization and high-level microarchitectural behavior.

2.1 Suite Overview of *SPEC CPU26*

Following *SPEC CPU17*, *SPEC CPU26* is organized into INT Rate, INT Speed, FP Rate, and FP Speed categories. As shown in Table 1, these categories contain 14, 13, 12, and 13 benchmarks, respectively, for a total of 52 benchmarks – 9 more than *SPEC CPU17*. Consistent with *SPEC CPU17*, *SPEC CPU26* continues to use C, C++, and Fortran implementations.

Compared with prior *SPEC CPU* generations, *SPEC CPU26* broadens application-domain coverage for future CPU research. All *SPEC CPU26* workloads are developed from scratch rather than carried over from previous suites. Counting Rate and Speed variants as a single workload, *SPEC CPU26* retains the names of only 3 out of 19 integer workloads from *SPEC CPU17* (*omnetpp*, *gcc*, *xz*)² and 6 out of 19 floating-point workloads (*cactus*, *fotonik3d*, *lbm*, *nab*, *namd*, *roms*); the remaining 16 integer and 13 floating-point workloads are newly introduced.

2.2 Performance Characterization

Platforms and metrics. We evaluate *SPEC CPU26* on nine systems spanning x86_64 and AArch64 ISAs, recent Intel, AMD, Ampere, and Nvidia processors, and diverse cache, core-count, and memory configurations (Table 2). All benchmarks are compiled with gcc -O3, and hardware counters are collected with Linux perf [73]. All systems use their default deployment configurations, including dynamic frequency scaling, since our analysis relies primarily on cycle- and instruction-normalized metrics. Table 3 lists the 19 metrics used in our analysis, covering IPC, cache/TLB behavior, branch prediction, frontend/backend stalls, instruction mix, and DRAM access intensity. This feature space follows established architectural characterization practice [62, 91, 98, 123, 128] and captures the major frontend, execution, memory-system, and control-flow bottleneck dimensions of general-purpose CPU workloads.

¹We present absolute performance metrics for a single representative platform to avoid the methodological artifacts that arise when aggregating hardware-dependent counters across different ISAs and microarchitectures.

²Although some *SPEC CPU26* workloads share names with predecessors, their codebases and inputs are entirely new; the same applies to floating-point workloads.

³All machines used in our experiments are dual-socket systems, and we report hardware configurations on a per-socket basis.

Table 1: Dynamic instruction count (billions), application domain, instruction mix, and IPC for each *SPEC CPU26* benchmark measured on a representative Intel Sapphire Rapids Platinum platform (CPU-C in Table 2)¹.

Benchmark	Domain	Icount	Loads	Stores	Branches	IPC
<i>SPEC CPU26</i> Integer Rate – 14 workloads						
706.stockfish_r	Games (Chess) [109]	6507	22.0	9.9	10.4	3.625
707.ntest_r	Games (Othello) [138]	2507	25.0	9.6	9.2	3.268
708.sqlite_r	Database [36]	1716	26.9	11.9	20.9	2.228
710.omnetpp_r	Network Sim. [134]	1583	31.9	17.5	20.5	2.103
714.cpython_r	Lang. Runtime [133]	1475	27.9	15.8	21.4	2.843
721.gcc_r	Compiler [124]	1503	28.0	11.3	21.8	0.551
723.llvm_r	Compiler [68]	1534	26.0	13.7	20.8	1.484
727.cppcheck_r	Static Analysis [77]	1286	22.5	9.6	26.7	2.228
729.abc_r	EDA [17]	1400	26.2	8.9	16.7	2.187
734.vpr_r	EDA (FPGA) [12]	1367	30.9	11.2	19.2	2.097
735.gem5_r	Arch. Sim. [15]	1659	30.2	14.7	20.9	2.068
750.sealcrypto_r	Cryptography [21]	3087	12.0	4.7	1.9	4.961
753.ns3_r	Network Sim. [108]	1432	29.4	16.8	22.2	2.230
777.zstd_r	Compression [32]	1817	22.2	9.0	13.3	1.911
<i>SPEC CPU26</i> Integer Speed – 13 workloads						
801.xz_s	Compression [102]	17757	22.2	7.4	14.4	1.008
807.ntest_s	Games (Othello) [138]	151005	20.8	7.8	6.7	3.460
817.flac_s	Compression	90970	17.6	2.3	4.4	4.156
821.gcc_s	Compiler [124]	109486	26.8	12.5	21.7	2.016
823.llvm_s	Compiler [68]	103105	22.0	11.8	23.1	1.896
827.cppcheck_s	Static Analysis [77]	90423	23.0	11.2	26.5	2.375
829.abc_s	EDA [17]	1433	25.1	11.7	18.1	0.858
834.vpr_s	EDA (FPGA) [12]	3117	30.7	11.1	19.4	1.863
835.gem5_s	Arch. Sim. [15]	2858	29.4	13.5	17.6	1.805
838.diamond_s	Bioinformatics [19]	146966	20.1	6.8	5.5	3.203
846.minizinc_s	CSP Solver [88]	5062	26.5	18.2	15.9	1.228
853.ns3_s	Network Sim. [108]	11053	28.9	14.3	21.0	1.662
854.graph500_s	Graph Analytics [81]	37168	36.2	0.9	25.7	1.539
<i>SPEC CPU26</i> Floating Point Rate – 12 workloads						
709.cactus_r	Physics (GR) [41]	1456	51.9	7.9	1.1	1.696
722.palm_r	Climate [78]	3272	39.0	9.1	5.0	3.187
731.ascenc_r	Img. Compress. [7]	2615	28.3	6.5	8.7	2.718
736.ocio_r	Image Process. [136]	2484	24.2	7.5	9.8	3.269
737.gmsh_r	Mesh Generation [38]	1086	29.2	12.1	17.2	1.585
748.flightdm_r	Aerospace [11]	1721	29.5	14.2	18.8	3.071
749.fotonik3d_r	Photonics [5]	1291	36.8	13.7	1.8	0.785
765.roms_r	Ocean Modeling [113]	2738	34.8	8.4	7.3	1.830
766.femflow_r	Fluid Dynamics [65]	5012	34.9	20.1	6.9	3.265
767.nest_r	Neuroscience [39]	1848	33.5	12.3	14.0	2.844
772.marian_r	NLP [61]	6389	8.7	1.3	3.0	3.953
782.lbm_r	Fluid Dynamics [130]	2236	21.2	10.9	0.7	1.241
<i>SPEC CPU26</i> Floating Point Speed – 13 workloads						
800.pot3d_s	Astrophysics [29]	7603	34.8	8.1	9.5	0.754
803.sph_exa_s	HPC (SPH) [44]	64626	24.8	3.4	11.2	2.465
809.cactus_s	Physics (GR) [41]	29190	51.9	8.1	1.6	1.338
811.tealeaf_s	Physics (Heat) [80]	40570	20.2	4.9	8.8	1.617
816.nab_s	Mol. Dynamics [75]	67717	31.2	5.7	11.9	2.441
820.cloverleaf_s	Physics (Hydro) [101]	25781	33.2	4.7	5.9	1.349
822.palm_s	Climate [78]	48883	38.2	8.9	6.4	1.920
849.fotonik3d_s	Photonics [5]	17777	56.0	9.8	2.7	0.955
857.namd_s	Mol. Dynamics [99]	168881	26.4	6.6	2.2	3.929
865.roms_s	Ocean Modeling [113]	28484	34.7	8.6	7.9	1.574
867.nest_s	Neuroscience [39]	66774	30.1	9.3	14.7	1.790
872.marian_s	NLP [61]	65980	10.8	2.8	3.8	3.258
881.neutron_s	Nuclear Physics [100]	33545	25.8	11.3	9.1	1.204

Table 1 reports dynamic instruction count, instruction mix, and IPC for each *SPEC CPU26* benchmark measured on CPU-C. Across the suite, dynamic counts reach trillions of instructions. Speed workloads are especially long-running, executing ~19× more instructions than FP Rate and ~29× more than INT Rate on average. Relative to *SPEC CPU17*, *SPEC CPU26* increases dynamic instruction count by 1.22×, 1.15×, 17.86×, and 2.67× for INT Rate, FP

Table 2: Hardware configurations of the nine machines (from Intel, AMD, Ampere, and Nvidia) used in the experiments³.

Machines	CPU-A	CPU-B	CPU-C	CPU-D	CPU-E	CPU-F	CPU-G	CPU-H	CPU-I
Vendor & Model	Intel Skylake	Intel Icelake	Intel Sapphire Rapids	Intel Sapphire Rapids	AMD Milan	AMD Genoa	AMD Turin	Ampere Altra	Nvidia Grace
Release Year	2017	2021	2023	2023	2021	2022	2024	2020	2023
Core	Platinum 8160	Platinum 8380	Platinum 8468	MAX 9480	EPYC 7763	EPYC 9454	EPYC 9555	Neoverse-N1	Neoverse-V2
# of Cores	24	40	48	56	64	48	64	80	72
L1-D/I cache	32/32KB	48/32KB	48/32KB	48/32KB	32/32KB	32/32KB	48/32KB	64/64KB	64/64KB
L2-cache	1MB	1.25MB	2MB	2MB	512KB	1MB	1MB	1MB	1MB
L3-cache/core	1.38MB	1.5MB	2.19 MB	2MB	4MB	4MB	4MB	0.4MB	1.63MB
DIMM	6× 16GB DDR4	8× 16GB DDR4	8× 96GB DDR5	4× 16GB HBM2e	8× 16GB DDR4	12× 16GB DDR5	12× 32GB DDR5	8× 16GB DDR4	224GB 32-Channel LPDDR5X
	2666 MT/s	3200 MT/s	4800 MT/s	3200 MT/s	3200 MT/s	4800 MT/s	6000 MT/s	3200 MT/s	8533 MT/s

Table 3: Performance metrics used in our analysis.

Category	Perf Metrics
Inst/Cycle	IPC
Cache	L1I\$ MPKI, L1D\$ MPKI, L2\$ MPKI, L3\$ MPKI
TLB	L1 iTLB MPMI, L1 dTLB MPMI, L2 TLB MPMI
Branch Predictor	Branch MPKI
Pipeline	Frontend/Backend stall %
Inst Mix	Kernel%, User%, Load%, Store%, Branch%, FP%, Vector%
DRAM	Mem access (bytes per cycle)

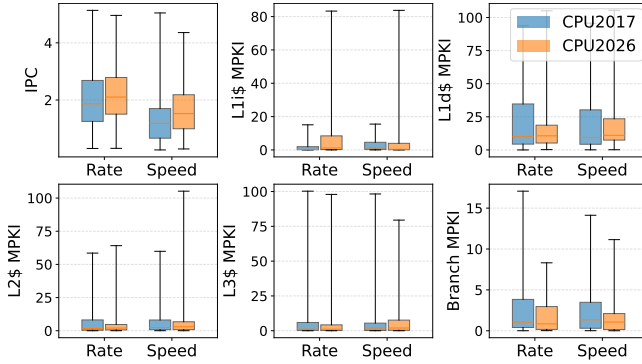


Figure 1: Key performance metric distributions comparing *SPEC CPU26* and *SPEC CPU17*, aggregated across all nine evaluated platforms. *SPEC CPU26* substantially broadens L1I\$ miss coverage while eliminating branch outliers.

Rate, INT Speed, and FP Speed, respectively. This growth reflects modern processor scaling, especially higher core counts in Speed runs [4, 13, 31, 86, 89, 92, 117, 119], but also increases detailed simulation cost [15, 42, 63, 91, 96, 115].

Figure 1 compares *SPEC CPU26* and *SPEC CPU17* using IPC and cache/branch MPKI distributions across all Rate and Speed workloads on the nine evaluated platforms. We aggregate data across systems to capture suite-wide behavioral coverage, treating cross-vendor MPKI values as indicative stress levels rather than strict microarchitectural equivalents. Compared with *SPEC CPU17* Rate, *SPEC CPU26* Rate exhibits much higher L1I\$ pressure (median 3.7×, mean 4.9×), with the high tail driven by 709.cactus_r, 753.ns3_r, and 723.llvm_r, while reducing mean branch MPKI by 42%. For Speed, *SPEC CPU17* has higher median L1I\$ MPKI (~1.18 vs. ~0.39), but *SPEC CPU26* has a wider upper tail driven by 809.cactus_s, 853.ns3_s, and 835.gem5_s. The wider branch and L3\$ MPKI ranges in *SPEC CPU17* are instead dominated by legacy outliers such as

Table 4: Representative subset per group and accuracy.

Group	Subset workloads	Accuracy
int_rate	750.sealcrypto_r, 735.gem5_r, 721.gcc_r, 708.sqlite_r	99.90%
fp_rate	749.fotonik3d_r, 709.cactus_r, 737.gmsh_r, 767.nest_r	99.07%
int_speed	807.ntest_s, 854.graph500_s, 823.llvm_s, 829.abc_s, 835.gem5_s	96.61%
fp_speed	867.nest_s, 816.nab_s, 809.cactus_s, 800.pot3d_s, 849.fotonik3d_s	96.40%

520.omnetpp_r and 505.mcf_r; 505.mcf_r is not included in *SPEC CPU26*, and 710.omnetpp_r is a fundamentally new workload despite sharing the name.

Takeaway: *SPEC CPU26* increases instruction volume and broadens behavioral coverage, especially by introducing new high-footprint workloads that expand L1I\$ stress while reducing dependence on several *SPEC CPU17* outliers.

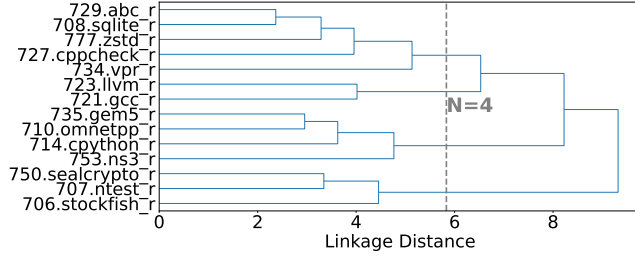
3 Representative Workloads in *SPEC CPU26*

This section uses similarity analysis to identify compact representative subsets of *SPEC CPU26*.

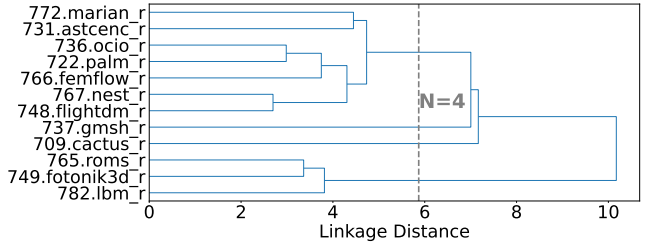
Similarity analysis methodology. We quantify workload similarity using a standard three-stage pipeline from prior benchmark studies [91, 98, 103]. First, we normalize all performance metrics so that differences reflect workload behavior rather than metric scale or machine-specific magnitude. Second, we apply Principal Component Analysis (PCA) to the cross-platform feature matrix, where each of the 19 metrics on each of the nine platforms is treated as a distinct feature, yielding 171 dimensions before reduction. PCA removes correlated dimensions while preserving dominant behavioral variation [91, 98, 103]. Finally, we apply hierarchical clustering in the PCA-reduced space and use linkage distance to quantify workload similarity and guide representative-subset selection.

Representative subsets. Using the hierarchical clusters, we derive compact workload subsets that preserve the behavioral diversity of the full suite. We cut each dendrogram at a fixed linkage distance to form behaviorally similar groups; Figure 2 illustrates this process for the INT/FP Rate/Speed suites in *SPEC CPU26*. Within each group, we select the medoid workload, i.e., the workload with the minimum average distance to other group members in PCA space.

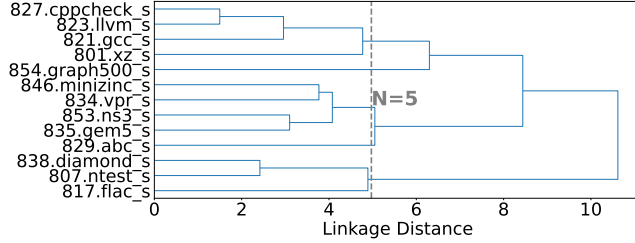
To quantify subset fidelity, we define subset *accuracy* as $1 - \text{err}$, where $\text{err} = |\text{GM}(\text{subset}) - \text{GM}(\text{suite})| / \text{GM}(\text{suite})$, and $\text{GM}(\cdot)$ is the SPEC geometric-mean score. Table 4 summarizes the selected subsets for all four *SPEC CPU26* groups using the 19-metric feature space described in Table 3. To evaluate metric-selection sensitivity, Table 5 repeats the subset analysis with cumulative metric groups



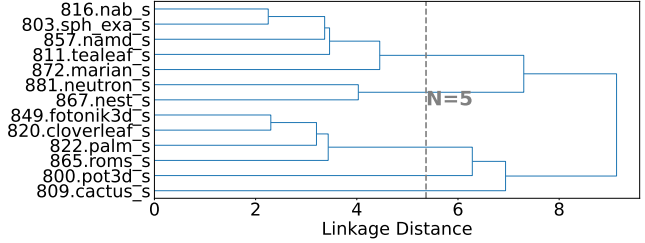
(a) *SPEC CPU26 INT Rate.*



(b) *SPEC CPU26 FP Rate.*



(c) *SPEC CPU26 INT Speed.*



(d) *SPEC CPU26 FP Speed.*

Figure 2: Dendrogram showing similarity between *SPEC CPU26* workloads; shorter linkage distance indicates higher similarity.

Table 5: Cumulative metric group subset accuracy. We used all 19 metrics for comprehensive performance characteristic coverage.

Metric group added	# of metrics	INT rate	FP rate	INT speed	FP speed	Avg
Inst/Cycle	1	97.05%	89.06%	99.22%	98.65%	95.99%
+Cache	5	99.90%	85.18%	98.43%	96.40%	94.98%
+TLB	8	99.99%	87.79%	99.93%	98.85%	96.64%
+Branch Predictor	9	99.99%	91.76%	98.24%	96.87%	96.72%
+Pipeline	11	97.93%	86.17%	99.00%	96.40%	94.88%
+Inst Mix	18	99.99%	99.07%	96.61%	96.40%	98.02%
+DRAM (full)	19	99.90%	99.07%	96.61%	96.40%	97.99%

Table 6: Dominant metrics per PC (top 4 by |mean loading|).

PC	Top 4 metrics (value in parens = mean of loadings over machines)
PC1	L3\$_{MKPI}\$ (+0.13), ipc (-0.11), L1D\$_{MKPI}\$ (+0.09), Backend (+0.09)
PC2	BranchInst (+0.14), ipc (-0.13), StoreInst (+0.13), Branch_MPKI (+0.10)
PC3	VectorInst (-0.10), Mem_Access (+0.09), FLOPs (-0.09), KernelInst (-0.08)
PC4	L1I\$_{MKPI}\$ (+0.16), BranchInst (-0.12), Branch_MPKI (-0.11), KernelInst (+0.10)
PC5	LoadInst (+0.16), L1D\$_{MKPI}\$ (+0.15), KernelInst (-0.10), UserInst (+0.10)
PC6	StoreInst (+0.20), Branch_MPKI (-0.15), L1D\$_{MKPI}\$ (-0.06), Mem_Access (+0.06)
PC7	BranchInst (+0.16), Branch_MPKI (-0.14), L2\$_{MKPI}\$ (-0.11), L1D\$_{MKPI}\$ (+0.08)
PC8	StoreInst (-0.12), L1I\$_{MKPI}\$ (+0.12), Branch_MPKI (-0.11), L2\$_{MKPI}\$ (+0.09)

from Table 3. Accuracy improves as more bottleneck dimensions are added, with the full 19-metric space achieving the highest average fidelity. Although memory access adds limited marginal gain because related pressure is partly captured by cache, TLB, and stall metrics, we retain it as a direct DRAM-pressure signal and for consistency with prior characterization practice [62, 91, 98, 123, 128].

Takeaway: Compact subsets of 4-5 workloads per group achieve 96.4-99.9% accuracy, substantially reducing evaluation cost while preserving representative behavior.

4 *SPEC CPU26* vs. *SPEC CPU17*, DCPerf, MLPerf

This section positions *SPEC CPU26* relative to its predecessor *SPEC CPU17* and two production-driven suites, DCPerf and MLPerf.

DCPerf targets hyperscale datacenter services with large instruction footprints and high frontend pressure [128], whereas MLPerf Inference emphasizes ML-centric tensor/vector and floating-point behavior under standardized model, dataset, and accuracy rules [106]. Since *SPEC CPU17* primarily consists of standalone general-purpose CPU programs, we ask whether *SPEC CPU26* updates this general-purpose baseline toward modern datacenter-adjacent behavior, while still remaining distinct from domain-specific suites such as DCPerf and MLPerf. We analyze workload-level similarity (§ 4.1) and compare detailed microarchitectural metrics (§ 4.2).

4.1 Similarity Analysis

We cluster *SPEC CPU26* Rate, *SPEC CPU17* Rate, DCPerf, and MLPerf workloads using the methodology from § 3; Figure 4 visualizes the resulting behavioral space. Mixed *SPEC CPU26*/*SPEC CPU17* clusters indicate overlap in general-purpose behavior, while isolated branches highlight domain-specific or outlier behavior.

Figure 4 shows that *SPEC CPU26* provides meaningful proximity to DCPerf without replacing it. DCPerf workloads have linkage distances of 1-2 to several *SPEC CPU26* workloads, indicating that *SPEC CPU26* captures some datacenter-relevant microarchitectural properties within its broader general-purpose scope, while DCPerf continues to represent service-specific extremes. In contrast, MLPerf workloads remain farther from both *SPEC CPU26* and *SPEC CPU17* (linkage distance ≥ 3 , up to 8), reflecting a different execution regime dominated by vector-heavy, data-parallel inference kernels. Thus, the MLPerf distance is not a coverage failure; rather, it bounds *SPEC CPU26*’s scope as a general-purpose CPU suite rather than a specialized accelerator-oriented inference suite.

Figure 3 and Table 6 further interpret the top eight principal components. The projections show that *SPEC CPU26* avoids several strong *SPEC CPU17* outliers, such as 505.mcf_r and 520.omnetpp_r, yielding more stable suite-level behavior. MLPerf separates primarily along vector/FLOP and memory-intensity dimensions, whereas

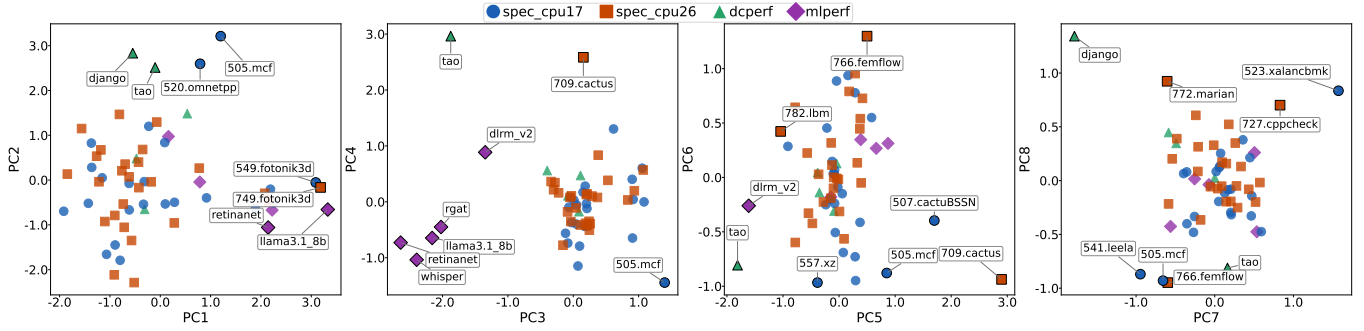


Figure 3: Principal Component (PC) comparison across four benchmark suites. The top 8 PCs capture 84% of total variability. Outlier and unique workloads are identified by distant PC values, plotting far from the central cluster. Notably, 505.mcf_r is an extreme outlier in *SPEC CPU17* and is not included in *SPEC CPU26*. Furthermore, DCPerf (tao, django) exhibits unique microarchitectural signatures on PC3/PC4 and PC7/PC8, while MLPerf workloads are primarily distinguished by PC3.

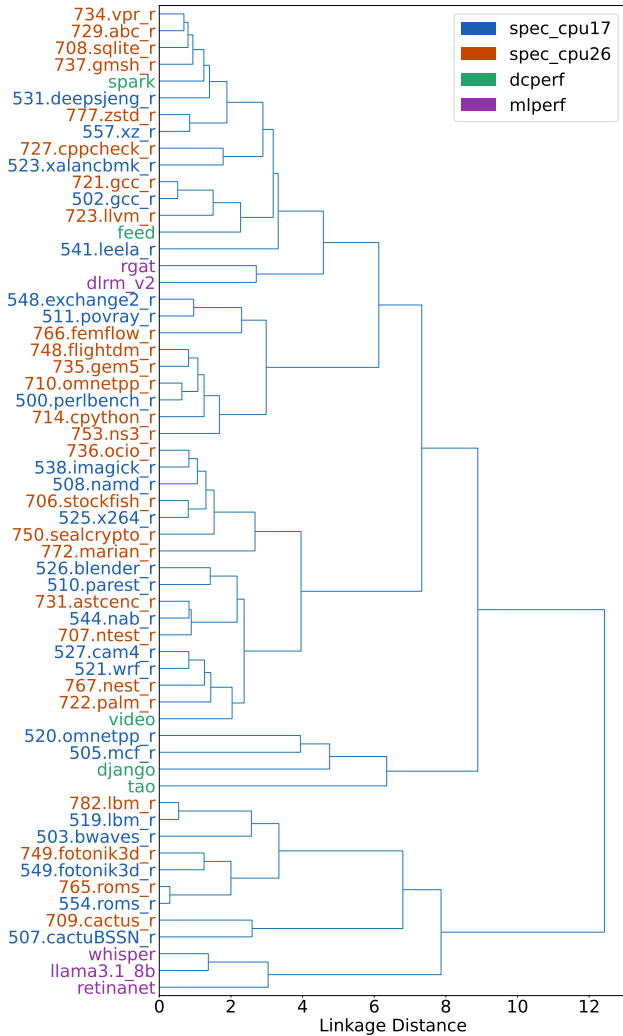


Figure 4: Dendrogram showing DCPerf and MLPerf have distinct behaviors compared to *SPEC CPU17*/*SPEC CPU26*.

DCPerf separates through frontend and service-style behavior (e.g., tao along PC3/PC4 and django along PC7/PC8) [128]. Still, *SPEC CPU26* bridges the DCPerf gap more closely than *SPEC CPU17*: for

example, 709.cactus_r reaches L1I\$ stress levels comparable to some DCPerf workloads, while remaining distinct on other dimensions such as frontend stalls (PC8).

Takeaway: *SPEC CPU26* contains fewer extreme outliers than *SPEC CPU17*, yielding more stable suite-level behavior. Meanwhile, *SPEC CPU26* moves closer to datacenter-like behavior represented by DCPerf while remaining clearly separated from highly vector-centric MLPerf, defining its modern general-purpose scope.

4.2 Detailed Microarchitectural Analysis

Beyond similarity, we compare *SPEC CPU26*, *SPEC CPU17*, DCPerf, and MLPerf on CPU-C for raw metric values (the trend across all nine machines is consistent). Figure 5 summarizes compute intensity, cache and branch behavior, and address-translation pressure. *Compute and vector intensity:* *SPEC CPU17* and *SPEC CPU26* impose broadly similar scalar compute pressure to DCPerf. For INT Rate, *SPEC CPU26* slightly increases the fraction of non-load/store/branch instructions over *SPEC CPU17* (43.21% vs. 39.71%, or 1.09×), close to DCPerf (42.51%); FP Rate shows a similar overall compute-intensity range. MLPerf, however, remains clearly distinct in vector instruction stress: its vector share reaches 29.97%, far above both *SPEC CPU17* and *SPEC CPU26*. Thus, *SPEC CPU26* is not primarily a vector-throughput stress test; its main value lies in frontend, cache, control-flow, and translation behavior.

Frontend and translation pressure: The largest differences between *SPEC CPU17* and *SPEC CPU26* appear in instruction-cache and translation behavior. For L1I\$ MPKI, *SPEC CPU26* rises by 2.53× over *SPEC CPU17* in INT Rate and 3.26× in FP Rate, substantially increasing frontend pressure while still remaining below DCPerf extremes. L1 iTLB pressure also increases, from 1.97 to 4.10 MPKI in INT Rate (2.08×) and from 1.03 to 1.61 in FP Rate (1.56×). L1 dTLB pressure similarly rises from 49.32 to 61.23 MPKI in INT Rate (1.24×) and from 10.25 to 16.98 in FP Rate (1.66×). These results show that *SPEC CPU26* expands frontend and translation stress relative to *SPEC CPU17*, but does not reach the deeper instruction footprints and service-style translation behavior of DCPerf.

Cache and branch behavior: Data-cache changes are more moderate. L1D\$ MPKI increases by 1.15× in INT Rate and 1.11× in FP Rate, while L2\$ MPKI changes little: INT rises modestly from 2.14 to 2.47, and FP is nearly unchanged (4.37 vs. 4.41). Both SPEC suites remain

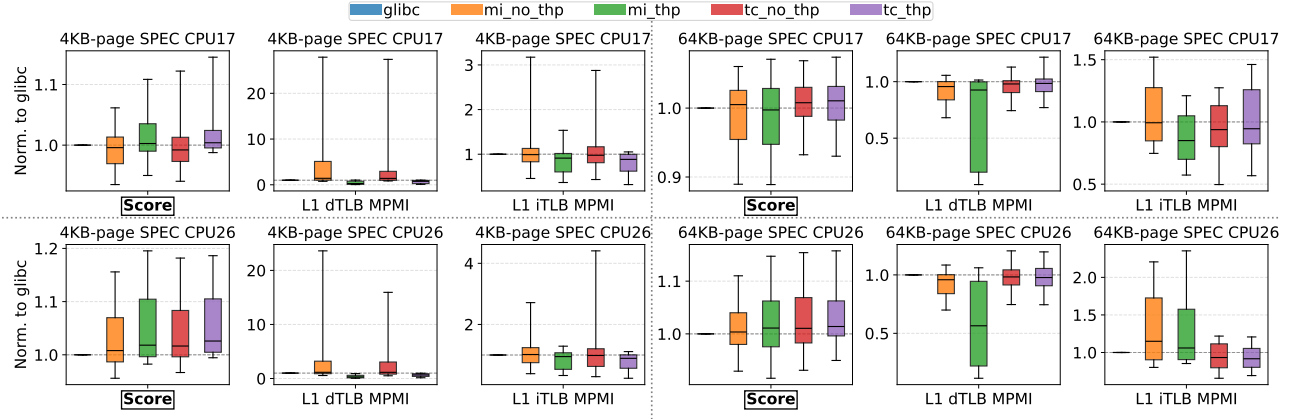
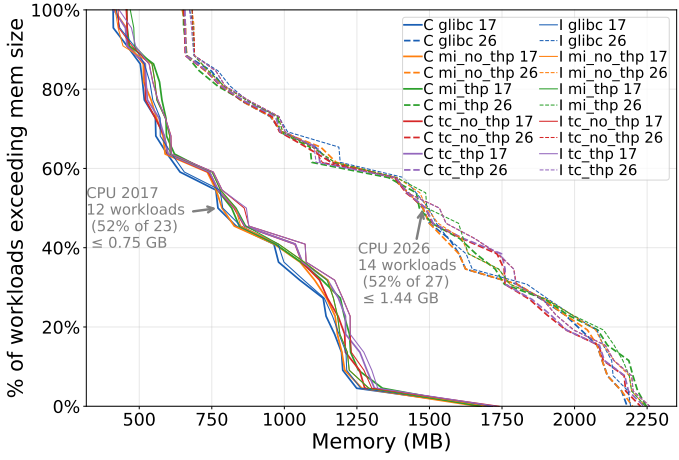
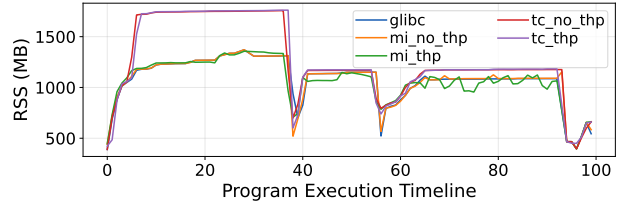


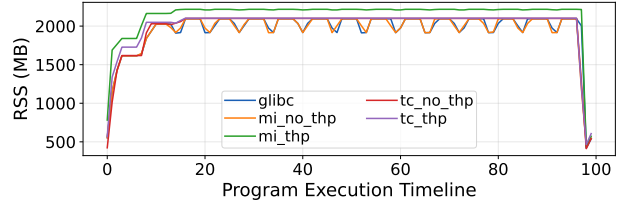
Figure 6: Performance impact of alternative memory allocators with/without THP across *SPEC CPU26* and *CPU2017 Rate* workloads (per-workload data in Appendix). The distributions show performance normalized to the default system *glibc* allocator. *tc*malloc and *mimalloc* consistently improve performance by reducing TLB misses, while THP provides workload- and machine-dependent benefits.



(a) Complementary CDF of all Rate workloads.



(b) RSS over time for 721.gcc_r (on CPU-C).



(c) RSS over time for 709.cactus_r (on CPU-C).

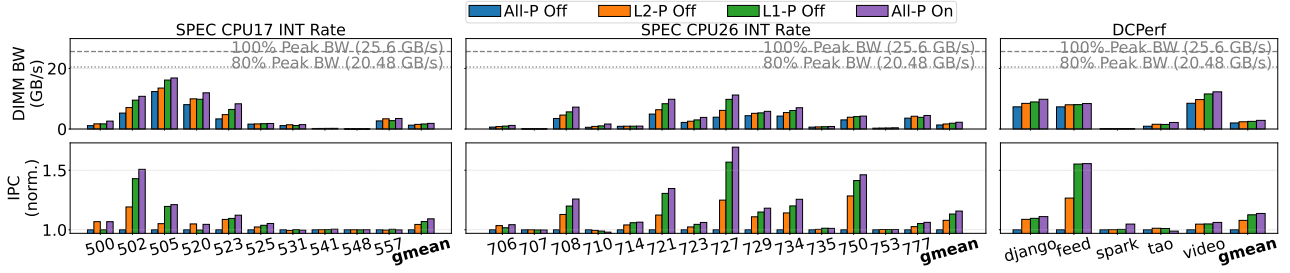
Figure 7: RSS for CPU2026/CPU2017 Rate (per-workload data in Appendix) on CPU-C (x86_64) and CPU-I (AArch64). While suite-level footprints are similar, allocator choice can significantly alter per-workload RSS.

SPEC CPU17 respond to page size and transparent huge pages (THP), we analyze seven platforms: five x86_64 servers with 4KB base pages and two AArch64 servers with 64KB base pages. We exclude the oldest (CPU-A) and newest (CPU-G) x86_64 systems to better align hardware generations with the AArch64 nodes. The microarchitectural impact of page size can be obscured by allocator behavior, since the default *glibc* allocator does not consistently back allocations with THPs [152]. To reduce this bias, we also evaluate *mimalloc* and *tc*malloc, which explicitly support and manage huge pages [50, 71]. Figure 6 aggregates per-workload performance distributions within each platform group.

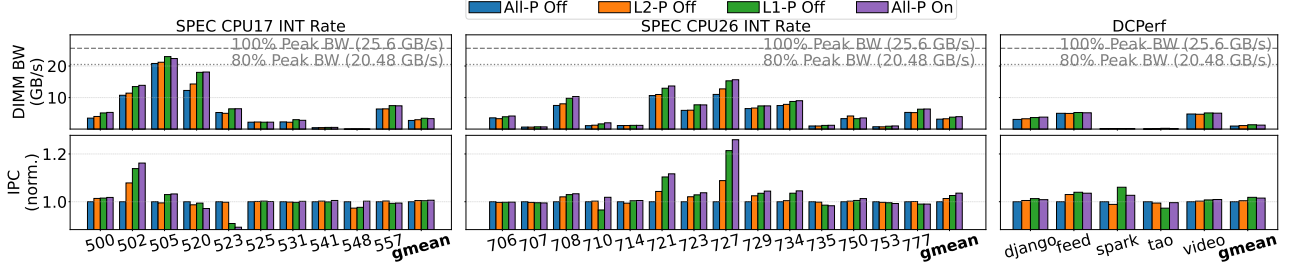
On 4KB-page systems, exposing proper THP behavior via these allocators reveals noticeable suite-level sensitivity to translation overheads. Relative to the baseline *glibc*, migrating to THP-aware allocators improves the geometric-mean *SPEC CPU26* score by 3.1-6.7% and *SPEC CPU17* by 0.0-5.5% (averaged across the five 4KB platforms), with individual microarchitectures seeing *SPEC CPU26*

gains up to 12.7%. These performance improvements confirm that 4KB-page systems suffer from translation bottlenecks that proper huge-page backing can alleviate. Specifically, *tc*malloc with THP reduces L1-dTLB and L1-iTLB MPMI by ~46.8% and ~17.0% respectively on average, reaching up to 65.3% and 56.2% reductions on the most TLB-sensitive architectures.

In contrast, servers utilizing a 64KB base page size natively bypass these translation bottlenecks, rendering THP tuning largely irrelevant. Across the two 64KB-page AArch64 platforms, enabling THP-aware allocators alters suite-level *SPEC CPU26* scores by a negligible -0.1% to 3.0%. While THP provides TLB benefits on 4KB systems, it can actually degrade performance on 64KB systems for certain workloads (e.g., 507.cactuBSSN_r in *SPEC CPU17* and 709.cactus_r in *SPEC CPU26* under *mimalloc*). Because the larger base page natively mitigates translation overheads, the fragmentation and metadata overheads of THP become more prominent.



(a) Results on CPU-B. Higher IPC comes from better DIMM bandwidth utilization.



(b) Prefetcher efficiency drops on CPU-E compared with CPU-B as more cores (64 vs. 40) share the same DIMM bandwidth.

Figure 8: IPC and DIMM bandwidth across prefetcher configurations on CPU-B and CPU-E.

Working Set Size. In addition to performance, page size can further impact the workload memory consumption. We compare memory consumption using resident set size (RSS) – a practical proxy for working set size (WSS) [43] – for the Rate workloads (Figure 7a). Across both page-size groups (Figure 7a), *SPEC CPU26* exhibits a consistently larger footprint (median ~ 1.4 GB vs. ~ 0.8 GB; max ~ 2.2 GB vs. ~ 1.7 GB). A few *SPEC CPU26* workloads share names with predecessors but their codebases and inputs are entirely new, so we treat cross-suite comparisons as indicative rather than direct continuations. Within *SPEC CPU26*, six workloads exceed 2.0GB – 706.stockfish_r, 722.palm_r, 734.vpr_r, 709.cactus_r, 782.lbm_r, and 749.fotonik3d_r – whereas no *SPEC CPU17* Rate workload reaches that threshold. The shift is therefore driven primarily by newly introduced workloads with natively high memory demands and by enlarged inputs on same-named scientific benchmarks, not by moderate datacenter-oriented additions (708.sqlite_r, 714.cpython_r, 723.llvm_r, 753.ns3_r; median ~ 1.1 GB). Beyond suite averages, temporal RSS traces reveal strong allocator sensitivities (Figures 7b and 7c). For 721.gcc_r, peak RSS varies by up to ~ 0.4 GB across allocators, with trace swings exceeding 1GB during execution – behavior tied to huge-page management rather than steady-state working-set size. Conversely, 709.cactus_r sustains ~ 2 GB for most of execution; under mimalloc+THP, mean RSS rises by $\sim 10\%$ and post-run reclamation is delayed, highlighting fragmentation risks on already memory-intensive workloads.

Takeaway: (1) Base page size strictly dictates a system’s sensitivity to huge-page tuning. Native 64KB base pages effectively mitigate translation overheads out of the box. On 4KB-page systems, THP successfully reduces these translation overheads, but trades that efficiency for software fragmentation risks. (2) *SPEC CPU26* has a $\sim 1.4\times$ larger memory footprint than *SPEC CPU17*, which amplifies its sensitivity to page size and translation overheads for memory-intensive workloads.

5.1.2 Hardware Prefetchers. Modern CPUs rely on hardware prefetchers to mitigate memory latency at the expense of memory bandwidth [10, 54, 69, 87, 90]. Because modifying hardware prefetcher configurations requires privileged access, we conduct this evaluation on CPU-B (Intel Icelake) and CPU-E (AMD Milan), the platforms in our analysis permitting root-level hardware control. Crucially, evaluating both Intel and AMD microarchitectures ensures that our conclusions are not artifactual to a single vendor’s prefetcher implementation. We evaluate the impact of hardware prefetchers using *SPEC CPU17*, *SPEC CPU26*, and DCPerf.

Figure 8 compares IPC and per-channel memory bandwidth across four prefetcher configurations: *All-P Off*, *L2-P Off*, *L1-P Off*, and *All-P On*. On CPU-B, enabling all prefetchers improves the geometric-mean IPC by $1.06\times$ for *SPEC CPU17*, $1.15\times$ for *SPEC CPU26*, and $1.13\times$ for DCPerf. The L2 prefetcher provides a more significant IPC boost than the L1 prefetcher (e.g., $1.12\times$ vs. $1.07\times$ for *SPEC CPU26*). Conversely, on CPU-E, the geometric-mean IPC gains are heavily muted: $1.01\times$ for *SPEC CPU17*, $1.04\times$ for *SPEC CPU26*, and $1.02\times$ for DCPerf. This discrepancy is largely attributable to the higher core count on CPU-E (64 cores vs. 40 cores on CPU-B) sharing the same 3200 MT/s per-channel memory bandwidth, leaving less bandwidth headroom for speculative prefetch requests.

Despite these vendor and architectural differences, both platforms exhibit the same qualitative trend: prefetching yields substantial improvements only when sufficient memory bandwidth headroom exists. Furthermore, these gains remain highly workload-dependent; for instance, *SPEC CPU26* exhibits a significant upside in 727.cppcheck_r, where prefetching improves IPC by $1.69\times$.

SPEC CPU26 also excludes certain “stress-test” workloads present in *SPEC CPU17*, such as 505.mcf_r, which exert substantially higher pressure on the memory subsystem than typical datacenter applications [128]. These specific workloads can reach nearly 80% of the system’s peak bandwidth – approaching practical saturation [30] –

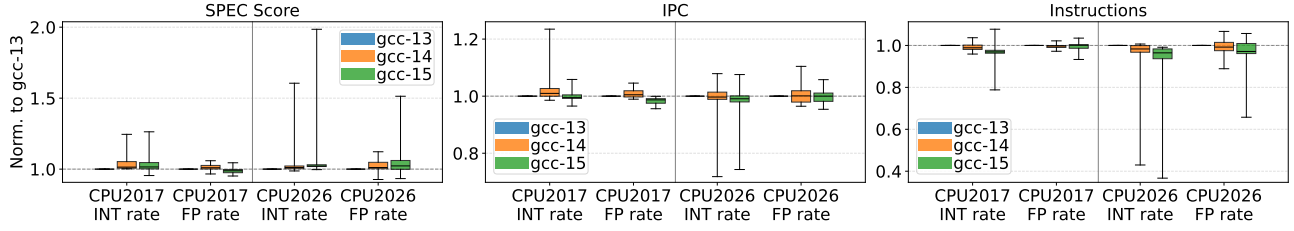


Figure 9: Performance distributions across gcc-13, gcc-14, and gcc-15 for *SPEC CPU26* and *SPEC CPU17* Rate workloads (per-workload data in Appendix). All results are averaged across our nine evaluated platforms and normalized to gcc-13. *SPEC CPU26* exhibits significantly greater performance sensitivity to compiler evolution, primarily driven by fluctuations in dynamic instruction count.

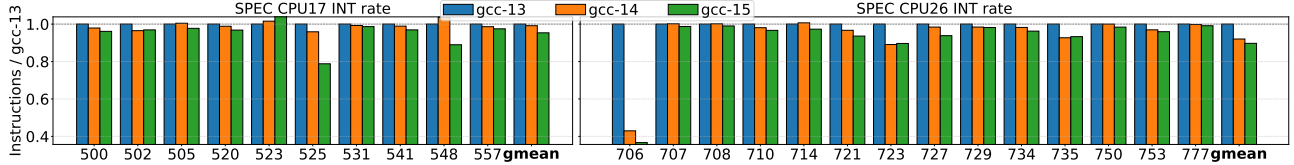


Figure 10: *SPEC CPU17* vs. *CPU26* inst count across compilers; *CPU26* is more compiler-sensitive (especially 706.stockfish_r).

while such extreme cases are often better represented by dedicated memory subsystem benchmarks such as *Mess* [16, 30].

Takeaway: *SPEC CPU26* successfully exposes prefetcher sensitivities without relying on extreme, unrealistic bandwidth stress-tests (e.g., 505.mcf_r in *SPEC CPU17*). Furthermore, cross-vendor analysis confirms that absolute prefetcher efficacy is highly contingent on the platform’s core-to-bandwidth ratio rather than the workload alone.

5.1.3 Compiler. *SPEC CPU26* can be used to evaluate the effectiveness of compilers, extending the long-standing role of its predecessor, *SPEC CPU17* [110, 111, 122]. This analysis addresses a fundamental question for architecture and compiler research: when toolchains evolve, do score changes reflect improved code generation or are they dominated by unrelated system effects? By incorporating a broader set of modern software applications, *SPEC CPU26* provides renewed opportunities for compiler optimization. Comparing compiler generations across both *SPEC CPU26* and *SPEC CPU17* helps quantify the actual impact of modern compiler advances on contemporary workloads.

We evaluate compiler sensitivity using gcc-13, gcc-14, and gcc-15. As shown in Figure 9 (distribution includes results of all nine platforms in Table 2), gcc-15 improves INT Rate performance over gcc-13 for both suites, with geomean speedups of 1.08 $\times$ on *SPEC CPU17* and 1.09 $\times$ on *SPEC CPU26*. For FP Rate, gcc-15 slightly regresses on *SPEC CPU17* (0.97 $\times$) but improves *SPEC CPU26* (1.07 $\times$). Examining IPC and instruction counts reveals a stark contrast: on *SPEC CPU26*, gains are driven primarily by a 17.7% geomean reduction in dynamic instruction count relative to gcc-13, despite a 3.3% decrease in IPC. In contrast, *SPEC CPU17* shows substantially weaker sensitivity, with instruction counts dropping by only 5.1% and IPC changing by just 0.1%. This reflects diminishing returns from a decade of compiler co-evolution with *SPEC CPU17*; the fresh workloads in *SPEC CPU26* offer renewed optimization headroom.

Figure 10 details per-workload instruction count of using different compilers (with results averaged across all nine platforms).

Certain *SPEC CPU26* programs respond robustly to modern optimizations; for instance, gcc-15 reduces the instruction count of 706.stockfish_r by up to 3 $\times$. Such stronger per-workload responses demonstrate that *SPEC CPU26* is more expressive than its predecessor, providing a more effective vehicle for demonstrating and evaluating compiler advances.

Takeaway: *SPEC CPU26* provides a fresh frontier for compiler innovation, exposing renewed optimization headroom that *SPEC CPU17* no longer offers due to a decade of saturation. Together, they establish a comprehensive evaluation regime: *SPEC CPU17* ensures stability for legacy applications, while *SPEC CPU26* prevents compiler stagnation by challenging toolchains with modern, high-footprint workloads.

5.2 Evaluating Architectural Designs

In this part, we show how *SPEC CPU26* can evaluate architectural design choices, including instruction-set comparisons (x86_64 vs. AArch64, § 5.2.1) and SoC-interconnect design trade-offs (§ 5.2.2).

5.2.1 ISA. We use *SPEC CPU26* to study ISA-level trade-offs, focusing on x86_64 versus AArch64. Because the nine platforms span diverse microarchitectures, program performance can be dominated by core/cache differences rather than ISA effects. We therefore use the per-copy retired instruction breakdown as the primary metric.

As shown in Figure 11, a key observation is that *SPEC CPU26* exhibits substantially higher cross-platform variation than *SPEC CPU17* even for INT Rate. Across all platforms, *SPEC CPU17* instruction counts span 0.78 $\times$ –1.22 $\times$ (standard deviation 5.9%), whereas *SPEC CPU26* expands the range to 0.72 $\times$ –1.51 $\times$ (standard deviation 10.8%). *SPEC CPU26* also contains more workloads with clear ISA-separated behavior; for example, 750.sealcrypto_r and 707.ntest_r retire ~20–30% fewer instructions on AArch64 than on x86_64, while *SPEC CPU17* has fewer cases with gaps of similar magnitude.

Takeaway: *SPEC CPU26* shows larger instruction-level variation across ISAs/platforms than *SPEC CPU17*, indicating broader ISA-sensitive coverage and making it better suited for ISA comparisons.

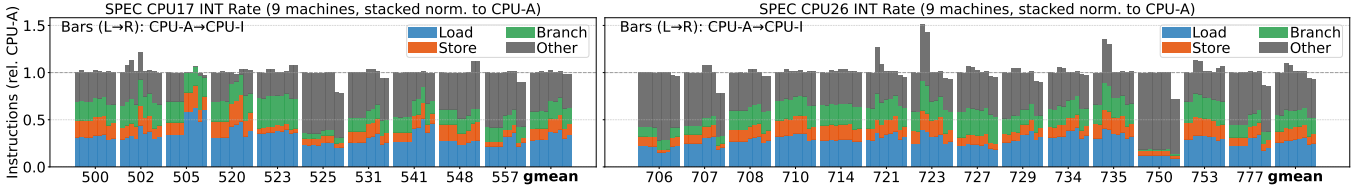


Figure 11: Per-copy retired-instruction distributions across nine machines (x86_64 and AArch64) for *SPEC CPU17* and *SPEC CPU26* INT Rate. *SPEC CPU26* shows greater cross-platform variation than *SPEC CPU17*.

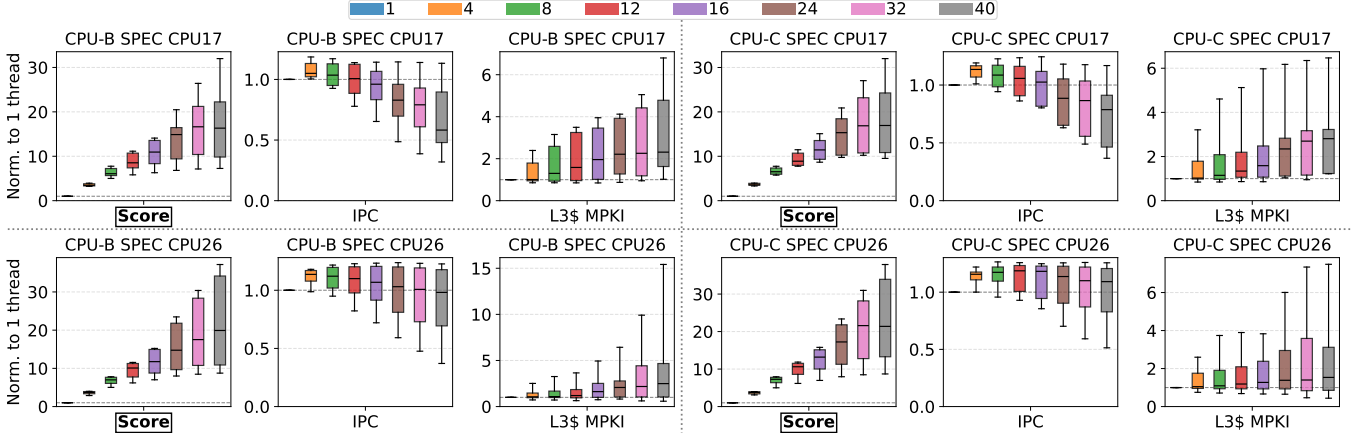


Figure 12: Scaling distributions (1–40 threads) for *SPEC CPU26*/*SPEC CPU17* Speed (per-workload data in Appendix) on CPU-B (monolithic, 40 cores) and CPU-C (chiplet, 4x12 cores). CPU-C scales better for most *SPEC CPU26* workloads due to a larger aggregate LLC, whereas *SPEC CPU17* more frequently exposes low-scaling communication bottlenecks that favor monolithic topologies.

5.2.2 SoC Interconnect. As processor core counts have increased substantially over the past decade [4, 13, 31, 86, 89, 92, 117, 119], multi-threaded workloads have become increasingly important for evaluating SoC interconnects. Their scalability is shaped by shared on-chip resources as well as cross-core communication, e.g., LLC slices, coherence protocols, and memory-system traffic, all of which become even more critical with the widespread adoption of chiplet-based CPUs [84, 118, 127]. Chiplets enable modular composition and improve performance scaling, but they can introduce non-uniform latencies or reduced bandwidth across chip boundaries [33, 66, 84, 118, 127, 135, 145]. We evaluate these trade-offs by comparing CPU-B (Intel Ice Lake, 40-core monolithic) and CPU-C (Intel Sapphire Rapids, 4x12-core chiplets).

Compared with *SPEC CPU17*, which includes 11 multi-threaded workloads in the Speed suite, *SPEC CPU26* includes 22, providing significantly broader coverage for modern many-core architectures where scalability beyond tens of threads is paramount. To isolate SoC interconnect effects, we sweep the thread counts of multi-threaded Speed workloads from 1 to 40. We present the Score, IPC, and L3\$ MPKI distributions in Figure 12. At 40 threads, the geometric-mean normalized Score reaches $15.31 \times / 16.31 \times$ (*SPEC CPU17*, CPU-B/CPU-C) versus $16.53 \times / 18.28 \times$ (*SPEC CPU26*, CPU-B/CPU-C). Moreover, *SPEC CPU26* continues to scale efficiently at higher thread counts (32→40: +9.5%/+10.3%), while *SPEC CPU17* exhibits early saturation (32→40: +3.1%/+5.0%). These trends demonstrate that *SPEC CPU26* features a modern workload profile that

remains highly scalable under high concurrency, making it a superior vehicle for stressing contemporary many-core topologies.

Chiplet vs. Monolithic. As additional context, CPU-C achieves 1.13× and 1.07× higher absolute geometric-mean performance than CPU-B on *SPEC CPU17* and *SPEC CPU26* INT Rate (per-copy), respectively. However, our scaling results are normalized to a single-thread baseline, factoring out raw core improvements to isolate cross-core interactions. In *SPEC CPU17*, we observe distinct monolithic advantages for a small subset of low-scaling workloads (classified in Table 7) that are highly sensitive to synchronization and serial cross-core communication overheads. For example, at 40 threads, 628.pop2_s achieves 11.72× on CPU-B but only 9.52× on CPU-C (a 1.23× monolithic advantage), while 621.wrf_s reaches 15.64× on CPU-B versus 12.35× on CPU-C (1.27×). Crucially, these performance gaps persist despite CPU-C maintaining a lower relative L3\$ MPKI increase (e.g., 628.pop2_s L3\$ MPKI at 40 threads scales by 1.97× on CPU-B vs. 1.34× on CPU-C). This demonstrates that non-uniform cross-chiplet coherence latencies can severely penalize older software design patterns that rely on tight synchronization, even when provided with superior cache capacity.

In contrast, *SPEC CPU26* exhibits a significantly more balanced scaling profile. CPU-B outpaces CPU-C for far fewer workloads, and the monolithic advantages are considerably narrower. The sole prominent outlier is the low-scaling 846.minizinc_s, which reaches 2.04× on CPU-B versus 1.68× on CPU-C at 40 threads (a 1.21× advantage). Beyond this single case, the performance deltas between the two topologies for the remaining low-scaling workloads fall

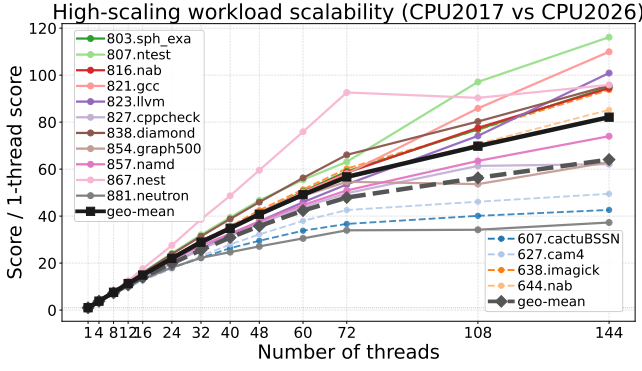


Figure 13: High-scalability workloads in *SPEC CPU26* continue to scale significantly better than *SPEC CPU17* past 40 threads into the ultra-many-core regime on CPU-I.

Table 7: Workload scalability classification by scaling (40-thread score > 20× 1-thread score = High-scaling).

Suite	Low-scaling	High-scaling
CPU2017	603.bwaves, 619.lbm, 621.wrf, 628.pop2, 649.fotonik3d, 654.roms, 657.xz	607.cactuBSSN, 627.cam4, 638.imagick, 644.nab
CPU2026	800.pot3d, 801.xz, 809.cactus, 811.tealeaf, 817.flac, 820.cloverleaf, 822.palm, 846.minizinc, 849.fotonik3d, 865.roms, 872.marian	803.sph_exa, 807.ntest, 816.nab, 821.gcc, 823.llvm, 827.cppcheck, 838.diamond, 854.graph500, 857.namd, 867.nest, 881.neutron

within a negligible ~1–2% margin. Consequently, while *SPEC CPU17* highlights unscalable cases where old synchronization models favor monolithic designs, *SPEC CPU26* demonstrates that modern code structures are robust against chiplet-induced latency penalties up to 40 threads. This proves that expanded L3 cache architectures successfully reduce cross-core communication overheads in realistic scenarios.

Scaling beyond 40 threads: Ultra-many-core behavior. To assess whether *SPEC CPU26* eventually succumbs to the same scalability plateaus as *SPEC CPU17* on larger topologies, we extend our evaluation to 144 threads on CPU-I (Nvidia Grace Superchip, 144 total cores in total of a dual-socket system). Figure 13 plots the high-scaling workloads identified in Table 7. *SPEC CPU26* continues to scale significantly better than *SPEC CPU17* past 40 threads: the geomean speedup of high-scaling workloads increases from 34.8× (*SPEC CPU26*) versus 30.9× (*SPEC CPU17*) at 40 threads, to 82.1× versus 64.1× at 144 threads, representing a relative scalability gain of +136% versus +107%. Consequently, *SPEC CPU26*’s higher high-scaling workload fraction provides a more rigorous and future-proof vehicle for evaluating next-generation ultra-many-core fabrics.

Takeaway: (1) *SPEC CPU26* provides a superior environment for SoC interconnect evaluation: it doubles the volume of multi-threaded Speed workloads and sustains high-concurrency scalability into ultra-many-core (144-thread) regimes where *SPEC CPU17* saturates. (2) The reduction of extreme, unscalable anomalies in *SPEC CPU26* represents a key improvement. Rather than relying on a few old software bottlenecks to differentiate hardware topologies, *SPEC CPU26* targets uniform, high-concurrency scaling characteristics that better reflect modern software systems.

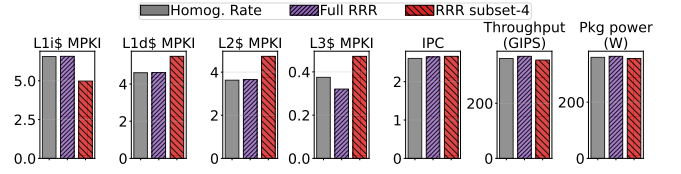


Figure 14: Full-suite RRR resembles homogeneous INT Rate profiles; targeted subsets shift the aggregate profile.

5.3 Creating Workload Proxies using RRR Mode

This section studies what *SPEC CPU26*’s Rolling Round-Robin (RRR) mode enables for heterogeneous CPU evaluation. In standard *SPEC Rate* execution, scheduling is homogeneous: all copies run the same benchmark concurrently before the suite advances to the next benchmark. For example, a 48-copy INT Rate run executes 48 copies of 706.stockfish_r together, then 48 copies of 707.ntest_r together, and so on. RRR changes only this scheduling structure: copies are staggered so different benchmarks run concurrently and rotate through the sequence, creating mixed-workload CPU stress tests from real benchmark programs. This resembles one microarchitectural aspect of multi-tenant systems, where different programs place different execution pressure on the same machine. However, RRR captures only profile composition, not full datacenter co-location behavior: it does not model latency-critical versus batch roles, tail latency, or application-level service-level objectives. We therefore use RRR as a reproducible proxy-construction tool, not as a replacement for end-to-end quality-of-service (QoS)-aware co-location benchmarks [25, 37, 45, 56].

RRR staggers N benchmarks across M cores so each core executes the same sequence with a phase offset, producing a time-rolled heterogeneous blend [53, 74, 95, 149]. We use RRR in two roles: as a full-suite execution mode over all INT Rate benchmarks, and as a composition primitive that mixes selected complementary workloads to bracket a target pressure point. Full-suite RRR preserves suite-level coverage, while targeted RRR proxies provide controlled stress points from selected benchmark programs rather than synthetic kernels. We validate this workflow in three steps: (a) compare standard homogeneous INT Rate with full-suite RRR INT Rate, (b) test whether targeted RRR proxies follow weighted solo profiles, and (c) quantify additional slowdown from simultaneous shared-resource interference.

Why targeted RRR proxies are needed. A natural first question is whether running RRR over the entire INT Rate suite already provides a useful heterogeneous stress point. Figure 14 compares standard homogeneous INT Rate, full-suite RRR INT Rate, and a four-workload RRR subset (750.sealcrypto_r, 735.gem5_r, 721.gcc_r, 708.sqlite_r from Table 4) on CPU-C. Full-suite RRR closely tracks standard homogeneous INT Rate – IPC 2.64 vs. 2.60, L1I\$ MPKI 6.58, throughput 371 vs. 363 giga instructions per second (GIPS), package power 363 vs. 359 W, and energy efficiency ~1.38–1.41 J/Ginstr – because averaging over the full suite smooths out workload-specific extremes. Thus, full-suite RRR is useful as a suite-level heterogeneous reference, but it is not designed to target a particular microarchitectural pressure point. Changing the workload mix can shift the aggregate profile, as shown by the four-workload subset’s

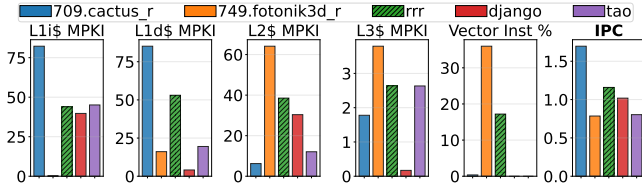


Figure 15: RRR proxy approaches DCPerf frontend pressure by mixing 709.cactus_r and 749.fotonik3d_r.

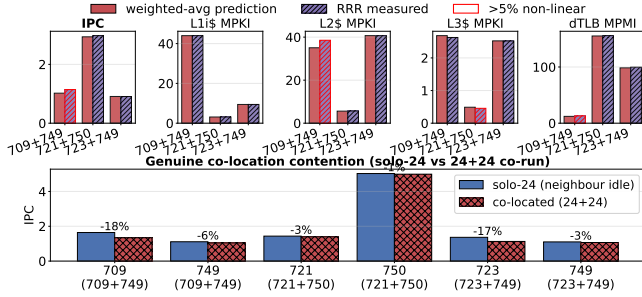


Figure 16: Top: RRR (hatched) vs. instruction/cycle-weighted prediction for three pairs; red outlines mark >5% departures. Bottom: co-location IPC (24+24 co-run vs. 24 solo).

different L1\$ MPKI (4.99 vs. 6.58), motivating targeted RRR proxies built from deliberately selected complementary workloads.

Bracketing a production point. *SPEC CPU26* workloads expose complementary behaviors: 709.cactus_r is frontend-dominated (82.3 L1\$ MPKI, 6.2 L2\$ MPKI), whereas 749.fotonik3d_r stresses the cache hierarchy and execution units (64.1 L2 MPKI, 36% vector) with negligible frontend pressure (0.25 L1\$ MPKI). Combining them in RRR therefore constructs a proxy that brackets a target pressure point. The resulting DCPerf-oriented proxy (Figure 15 running on CPU-C) falls between its constituents on every metric: L1\$ MPKI is 43.9, L2/L3 MPKI are 38.5/2.64, and IPC is 1.16. It also lands near DCPerf frontend pressure (43.9 vs. 39.7/45.1 for django/tao) with comparable IPC (1.16 vs. 1.02 for django).

To evaluate whether this construction principle extends beyond the original pair, we repeated the experiment with two additional workload pairs: 723.llvm_r + 749.fotonik3d_r and 721.gcc_r + 750.sealcrypto_r. Figure 16 shows that RRR profiles largely track weighted predictions from the solo endpoints: instruction-weighted for per-instruction metrics and cycle-weighted for cycle-normalized metrics (running on CPU-C). Each weight is the workload’s share of the rate’s denominator – retired instructions for MPKI, elapsed cycles for IPC. For example, 721.gcc_r + 750.sealcrypto_r closely matches the instruction-weighted dTLB prediction (156 measured vs. 155 predicted). The largest residuals occur for 709.cactus_r + 749.fotonik3d_r, where IPC and L2 MPKI exceed prediction by 12% and 10%, respectively. These moderate residuals show that weighted solo profiles are useful as first-order estimates, but may not be treated as exact linear interpolation. Together, the results support a practical RRR proxy-construction workflow: select *SPEC CPU26* workloads that bracket a target pressure point, compose them in RRR, estimate the aggregate behavior from weighted solo profiles, and measure residual deviations when required.

RRR proxies, co-location, and QoS limits. RRR exposes a heterogeneous aggregate profile, but its native outputs remain scores and hardware counters. It therefore cannot distinguish latency-critical and batch roles, report tail latency, or determine service-level objective violations. This illustrates why absolute *SPEC CPU* scores alone are insufficient for reasoning about multi-tenant service behavior. However, RRR remains useful for CPU architects: it generates reproducible pressure mixes that can be paired with separate interference measurements or application-level QoS studies. To quantify the interference component directly, we co-locate the two halves of each pair simultaneously (24+24 copies on the 48-core CPU-C) and compare against a 24-copy solo baseline. Co-location lowers per-workload IPC by up to 18% (709.cactus_r –18%, 749.fotonik3d_r –6%), while compute-bound 750.sealcrypto_r is largely unaffected (–1%), confirming that shared-resource interference is concentrated in memory-sensitive phases (Figure 16, bottom).

Takeaway: RRR can generate heterogeneous microarchitectural proxies by composing complementary *SPEC CPU26* workloads; across three pairs, the aggregate profile largely follows weighted endpoint predictions. Full-suite RRR preserves suite-level behavior but averages away targeted pressure points, so RRR is best viewed as a proxy-generation mechanism rather than a QoS benchmark; true co-location effects require separate interference or application-level measurements.

6 Conclusion

This paper presents the first comprehensive characterization of *SPEC CPU26* and revisits CPU-centric benchmarking in an era shaped by accelerators and ML workloads. Across nine platforms, we show that *SPEC CPU26* broadens coverage over *SPEC CPU17* with larger instruction volumes, higher memory footprints, and stronger pressure on modern bottlenecks, especially the instruction cache. Compact representative subsets of 4-5 workloads per group preserve full-suite behavior with high fidelity (96.4-99.9%). Cross-suite comparisons with *SPEC CPU17*, DCPerf, and MLPerf show that *SPEC CPU26* remains a general-purpose CPU benchmark: it is close to foundational datacenter workloads represented by DCPerf, yet clearly separated from highly vector-centric MLPerf inference. Our case studies further demonstrate practical value beyond score reporting, including system-configuration studies, architectural comparisons, and RRR-based proxy workload generation. Overall, *SPEC CPU26* provides a practical, forward-looking foundation for rigorous CPU evaluation while remaining complementary to domain-specific suites such as DCPerf and MLPerf.

Acknowledgment

The authors would like to thank the *SPEC CPU* Subcommittee for the opportunity to collaborate on the clustering analysis for the *SPEC CPU2026* program selection. We are grateful to the member companies – specifically AMD, Ampere, IBM, and Intel – for providing the necessary data. Special thanks go to John Henning, Mahesh Madhav, and Frédérique Silber-Chaussumier for their invaluable technical support and for facilitating the analysis. Additionally, we thank the Texas Advanced Computing Center (TACC) at the University of Texas at Austin for providing the computing resources that supported this work.

References

- [1] Siddharth Agarwal, Tianchen Wang, Jinghan Huang, Saksham Agarwal, and Nam Sung Kim. 2026. TiNA: Tiered Network Buffer Architecture for Fast Networking in Chiplet-based CPUs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 298–313.
- [2] Manoj Alwani, Han Chen, Michael Ferdman, and Peter Milder. 2016. Fused-layer CNN accelerators. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [4] Ampere. 2026. Ampere Processor Platforms. <https://amperecomputing.com/products/processors>.
- [5] Ulf Andersson, Min Qiu, and Ziyang Zhang. 2006. Parallel power computation for photonic crystal devices. *Methods and applications of analysis* 13, 2 (2006), 149–156.
- [6] Georgia Antoniou, Davide Bartolini, Haris Volos, Marios Kleanthous, Zhe Wang, Kleovoulos Kalaitzidis, Tom Rollet, Ziwei Li, Onur Mutlu, Yiannakis Sazeides, and Jawad Haj Yahya. 2024. Agile C-states: a core C-state architecture for latency critical applications optimizing both transition and cold-start latency. *ACM Transactions on Architecture and Code Optimization* 21, 4 (2024), 1–26.
- [7] ARM. 2026. The Arm ASTC Encoder, a compressor for the Adaptive Scalable Texture Compression data format. <https://github.com/ARM-software/astc-encoder>.
- [8] ARM. 2026. The world's most efficient agentic CPU. <https://www.arm.com/products/cloud-datacenter/arm-agi-cpu>.
- [9] Rachata Ausavarungrun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J Rossbach, and Onur Mutlu. 2018. Mosaic: Enabling application-transparent support for multiple page sizes in throughput processors. *ACM SIGOPS Operating Systems Review* 52, 1 (2018), 27–44.
- [10] Mohammad Bakhshalipour, Seydali Tabaeiaghdaei, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Evaluation of hardware data prefetchers on server processors. *ACM Computing Surveys (CSUR)* 52, 3 (2019), 1–29.
- [11] Jon Berndt. 2004. JSBSim: An open source flight dynamics model in C++. In *AIAA modeling and simulation technologies conference and exhibit*. 4923.
- [12] Vaughn Betz and Jonathan Rose. 1997. VPR: A new packing, placement and routing tool for FPGA research. In *International Workshop on Field Programmable Logic and Applications*. Springer, 213–222.
- [13] Ravi Bhargava and Kai Troester. 2024. AMD next-generation “Zen 4” core and 4th gen AMD EPYC server CPUs. *IEEE Micro* 44, 3 (2024), 8–17.
- [14] Tawhid Bhuiyan, Sumya Hoque, Angélica Aparecida Moreira, and Tanvir Ahmed Khan. 2026. Wax: Optimizing Data Center Applications With Stale Profile. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 2232–2248.
- [15] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 simulator. *ACM SIGARCH computer architecture news* 39, 2 (2011), 1–7.
- [16] F. Nisa Bostanci, Haocong Luo, Ataberk Olgun, Maria Makeenkova, Gerardo F. Oliveira, A. Giray Yaglikci, and Onur Mutlu. 2026. Cleaning up the Mess: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0. arXiv:2510.15744 [cs.AR] <https://arxiv.org/abs/2510.15744>
- [17] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*. Springer, 24–40.
- [18] James Bucek, Klaus-Dieter Lange, and Jóakim v. Kistowski. 2018. SPEC CPU2017: Next-generation compute benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 41–42.
- [19] Benjamin Buchfink, Klaus Reuter, and Hajk-Georg Drost. 2021. Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nature methods* 18, 4 (2021), 366–368.
- [20] Calin Cascaval, Evelyn Duesterwald, Peter F Sweeney, and Robert W Wisniewski. 2005. Multiple page size modeling and optimization. In *14th International Conference on Parallel Architectures and Compilation Techniques (PACT’05)*. IEEE, 339–349.
- [21] Hao Chen, Kim Laine, and Rachel Player. 2017. Simple encrypted arithmetic library-SEAL v2. 1. In *International conference on financial cryptography and data security*. Springer, 3–18.
- [22] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the potential of fpga-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems* 18, 1 (2024), 1–29.
- [23] Shuang Chen, Christina Delimitrou, and José F Martinez. 2019. Parties: Qos-aware resource partitioning for multiple interactive services. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 107–120.
- [24] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [25] Yue Cheng, Zheng Chai, and Ali Anwar. 2018. Characterizing co-located data-center workloads: An alibaba case study. In *Proceedings of the 9th Asia-Pacific Workshop on Systems*. 1–3.
- [26] Weiwei Chu, Xinfeng Xie, Jiecao Yu, Jie Wang, Amar Phanishayee, Chunqiang Tang, Yuchen Hao, Jianyu Huang, Mustafa Ozdal, Jun Wang, Vedanuj Goswami, Naman Goyal, Abhishek Kadian, Andrew Gu, Chris Cai, Feng Tian, Xiaodong Wang, Min Si, Pavan Balaji, Ching-Hsiang Chu, and Jongsoo Park. 2025. Scaling Llama 3 Training with Efficient Parallelism Strategies. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1703–1716.
- [27] Joel Coburn, Chunqiang Tang, Sameer Abu Asa, Neeraj Agrawal, Raviteja Chinta, Harish Dixit, Brian Dodds, Saritha Dwarakapuram, Amin Firoozshahian, Cao Gao, Kaustubh Gondkar, Tyler Graf, Junhan Hu, Jian Huang, Sterling Hughes, Adam Hutchin, Bhasker Jakka, Guoqiang Jerry Chen, Indu Kalyanaraman, Ashwin Kamath, Pankaj Kansal, Erum Kazi, Roman Levenstein, Mahesh Maddury, Alex Mastro, Siji Medaiyese, Pritesh Modi, Jack Montgomery, Satish Nadathur, Amit Nagpal, Ashwin Narasimha, Maxim Naumov, Eleanor Ozer, Jongsoo Park, Poorva Ramani, Harikrishna Reddy, David Reiss, Deboleena Roy, Sathish Sekar, Arushi Sharma, Pavan Shetty, Aravind Sukumar-Rajam, Eran Tal, Mike Tsai, Shreya Varshini, Richard Wareing, Olivia Wu, Xiaolong Xie, Jinghan Yang, Hangchen Yu, Tanmay Zargar, Zitong Zeng, Feixiong Zhang, Ajit Mathews, Xun Jiao, Jiyuan Zhang, Emmanuel Menage, Truls Edvard Stokke, and Mohammed Sourouri. 2025. Meta’s Second Generation AI Chip: Model-Chip Co-Design and Productionization Experiences. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1689–1702.
- [28] Guilherme Cox and Abhishek Bhattacharjee. 2017. Efficient address translation for architectures with multiple page sizes. *ACM SIGPLAN Notices* 52, 4 (2017), 435–448.
- [29] Cooper Downs, Jon A. Linker, Ronald M. Caplan, Emily I. Mason, Pete Riley, Ryder Davidson, Andres Reyes, Erika Palmerio, Roberto Lionello, James Turtle, Michal Ben-Nun, Miko M. Stulajter, Viacheslav S. Titov, Tibor Török, Lisa A. Upton, Raphael Attie, Bibhuti K. Jha, Charles N. Arge, Carl J. Henney, Gherardo Valori, Hanna Strecker, Daniele Calchetti, Dietmar Germerott, Johann Hirzberger, David Orozco Suárez, Julian Blanco Rodríguez, Sami K. Solanki, Xin Cheng, and Sizhe Wu. 2025. A near-real-time data-assimilative model of the solar corona. *Science* 388, 6753 (2025), 1306–1310.
- [30] Pouya Esmaili-Dokht, Francesco Sgherzi, Valéria Soldera Girelli, Isaac Boix-aderas, Mariana Carmin, Alireza Monemi, Adrià Armejach, Estanislao Mercadal, Germán Llort, Petar Radojković, Miquel Moreto, Judit Giménez, Xavier Martorell, Eduard Ayguadé, Jesus Labarta, Emanuele Confalonieri, Rishabh Dubey, and Jason Adlard. 2024. A mess of memory system benchmarking, simulation and application profiling. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 136–152.
- [31] Mark Evers, Leslie Barnes, and Mike Clark. 2022. The AMD next-generation “Zen 3” core. *IEEE Micro* 42, 3 (2022), 7–12.
- [32] Facebook. 2026. Zstandard - Fast real-time compression algorithm. <https://github.com/facebook/zstd>.
- [33] Yinxiao Feng and Kaisheng Ma. 2022. Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 121–126.
- [34] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, Adam Hutchin, Utku Diril, Krishnakumar Nair, Ehsan K. Aredestani, Martin Schatz, Yuchen Hao, Rakesh Komuravelli, Kunming Ho, Sameer Abu Asa, Joe Shajrawi, Kevin Quinn, Nagesh Sreedhara, Pankaj Kansal, Willie Wei, Dheepak Jayaraman, Linda Cheng, Pritam Chopda, Eric Wang, Ajay Bikumandla, Arun Karthik Sengottuvel, Krishna Thottempudi, Ashwin Narasimha, Brian Dodds, Cao Gao, Jiyuan Zhang, Mohammed Al-Sanabani, Ana Zehtabioskuie, Jordan Fix, Hangchen Yu, Richard Li, Kaustubh Gondkar, Jack Montgomery, Mike Tsai, Saritha Dwarakapuram, Sanjay Desai, Nili Avidan, Poorva Ramani, Karthik Narayanan, Ajit Mathews, Sethu Gopal, Maxim Naumov, Vijay Rao, Krishna Noru, Harikrishna Reddy, Prahlad Venkatapuram, and Alexis Bjorlin. 2023. Mtia: First generation silicon targeting meta’s recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [35] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam Sapek, Gabriel Weisz, Lisa Woods, Sitaram Lanka, Steven K. Reinhardt, Adrian M. Caulfield, Eric S. Chung, and Doug Burger. 2018. A configurable cloud-scale DNN processor for real-time AI. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1–14.

- [36] Kevin P Gaffney, Martin Prammer, Larry Brasfield, D Richard Hipp, Dan Kennedy, and Jignesh M Patel. 2022. SQLite: past, present, and future. *Proceedings of the VLDB Endowment* 15, 12 (2022).
- [37] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rath, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinisky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.
- [38] Christophe Geuzaine and Jean-François Remacle. 2009. Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities. *International journal for numerical methods in engineering* 79, 11 (2009), 1309–1331.
- [39] Marc-Oliver Gewaltig and Markus Diesmann. 2007. Nest (neural simulation tool). *Scholarpedia* 2, 4 (2007), 1430.
- [40] Abraham Gonzalez, Aasheesh Kolli, Samira Khan, Sihang Liu, Vidushi Dadu, Sagar Karandikar, Jichuan Chang, Krste Asanovic, and Parthasarathy Ranganathan. 2023. Profiling hyperscale big data processing. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–16.
- [41] Tom Goodale, Gabrielle Allen, Gerd Lanfermann, Joan Massó, Thomas Radke, Edward Seidel, and John Shalf. 2002. The cactus framework and toolkit: design and applications: invited talk. In *International conference on high performance computing for computational science*. Springer, 197–227.
- [42] Björn Gottschall, Silvio Campelo de Santana, and Magnus Jahre. 2023. Balancing accuracy and evaluation overhead in simulation point selection. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 43–53.
- [43] Darryl Gove. 2007. CPU2006 working set size. *ACM SIGARCH Computer Architecture News* 35, 1 (2007), 90–96.
- [44] Danilo Guerrero, Rubén M Cabezon, Jean-Guillaume Piccinali, Aurélien Cavelan, Florina M Ciorba, David Imbert, Lucio Mayer, and Darren Reed. 2018. Towards a mini-app for smoothed particle hydrodynamics at exascale. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 607–614.
- [45] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In *Proceedings of the international symposium on quality of service*. 1–10.
- [46] Faruk Guvenilir and Yale N Patt. 2020. Tailored page sizes. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 900–912.
- [47] Ranjan Hebbar SR and Aleksandar Milenković. 2019. SPEC CPU2017: Performance, event, and energy characterization on the core i7-8700K. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. 111–118.
- [48] John L Henning. 2002. SPEC CPU2000: Measuring CPU performance in the new millennium. *Computer* 33, 7 (2002), 28–35.
- [49] John L Henning. 2006. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News* 34, 4 (2006), 1–17.
- [50] Andrew Hamilton Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. 2021. Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 257–273.
- [51] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. Floptim: In-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture*. 802–815.
- [52] Koki Ishida, Ilkwon Byun, Ikki Nagaoka, Kosuke Fukumitsu, Masamitsu Tanaka, Satoshi Kawakami, Teruo Tanimoto, Takatsugu Ono, Jangwoo Kim, and Koji Inoue. 2020. SuperNPU: An extremely fast neural processing unit using superconducting logic devices. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 58–72.
- [53] Adam N Jacobvitz, Andrew D Hilton, and Daniel J Sorin. 2015. Multi-program benchmark definition. In *2015 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 72–82.
- [54] Akanksha Jain, Hannah Lin, Carlos Villavieja, Baris Kasikci, Chris Kennelly, Milad Hashemi, and Parthasarathy Ranganathan. 2024. Limoncello: Prefetchers for scale. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 577–590.
- [55] Rishabh Jain, Scott Cheng, Vishwas Kalagi, Vrushabh Sanghavi, Samvit Kaul, Meena Arunachalam, Kiwan Maeng, Adwait Jog, Anand Sivasubramaniam, Mahmut Taylan Kandemir, and Chita R. Das. 2023. Optimizing cpu performance for recommendation systems at-scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [56] Congfeng Jiang, Yitao Qiu, Weisong Shi, Zhefeng Ge, Jiwei Wang, Shenglei Chen, Christophe Cérin, Zujie Ren, Guoyao Xu, and Jiangbin Lin. 2020. Characterizing co-located workloads in alibaba cloud datacenters. *IEEE Transactions on Cloud Computing* 10, 4 (2020), 2381–2397.
- [57] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Proceedings of Machine Learning and Systems* 7 (2025).
- [58] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [59] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggione, Mair Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelman, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [60] Jowi Morales. 2026. Are we staring down the barrel of an AI-driven CPU shortage? <https://www.tomshardware.com/pc-components/cpus/cpus-are-cool-again-intel-and-amd-reporting-spikes-in-cpu-demand-due-to-agentic-ai-shortages-lisa-su-says-business-exceeded-expectations-while-intel-is-looking-at-long-term-agreements-with-potential-customers>.
- [61] Marcin Junczys-Dowmunt, Roman Grundkiewicz, Tomasz Dwojak, Hieu Hoang, Kenneth Heafield, Tom Neckermann, Frank Seide, Ulrich Germann, Alham Fikri Aji, Nikolay Bogoychev, André F. T. Martins, and Alexandra Birch. 2018. Marian: Fast neural machine translation in C++. In *Proceedings of ACL 2018, system demonstrations*. 116–121.
- [62] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. 2015. Profiling a warehouse-scale computer. In *Proceedings of the 42nd annual international symposium on computer architecture*. 158–169.
- [63] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, Qijing Huang, Kyle Kovacs, Borivoje Nikolic, Randy Katz, Jonathan Bachrach, and Krste Asanovic. 2018. FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.
- [64] Joshua Kim, Chaojie Zhang, Inigo Goiri, Christopher J Rossbach, and Jovan Stojkovic. 2026. PhaseWeave: Phase-Aware Execution on Heterogeneous Chiplet Architectures for Datacenters. (2026).
- [65] Martin Kronbichler, Dmytro Sashko, and Peter Munch. 2023. Enhancing data locality of the conjugate gradient method for high-order matrix-free finite-element implementations. *The International Journal of High Performance Computing Applications* 37, 2 (2023), 61–81.
- [66] Jaewon Kwon, Yongju Lee, Hongju Kal, Minjae Kim, Youngsok Kim, and Won Woo Ro. 2023. McCore: A Holistic Management of High-Performance Heterogeneous Multicores. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1044–1058.
- [67] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [68] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 75–86.
- [69] Jaekyu Lee, Hyesoon Kim, and Richard Vuduc. 2012. When prefetching works, when it doesn't, and why. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 1 (2012), 1–29.
- [70] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.
- [71] Daan Leijen, Benjamin Zorn, and Leonardo De Moura. 2019. Mimalloc: Free list sharding in action. In *Asian Symposium on Programming Languages and Systems*. Springer, 244–265.
- [72] Ankur Limaye and Tosiron Adegbija. 2018. A workload characterization of the spec cpu2017 benchmark suite. In *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 149–158.

- [73] Linux Kernel Community. 2026. Linux perf tool. https://perf.wiki.kernel.org/index.php/Main_Page.
- [74] Qiuyun Llull, Songchun Fan, Seyed Majid Zahedi, and Benjamin C Lee. 2017. Cooper: Task colocation with cooperative games. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 421–432.
- [75] Thomas J Macke and David A Case. 1998. Modeling unusual nucleic acid structures. ACS Publications.
- [76] Artemiy Margaritov, Siddharth Gupta, Rekai Gonzalez-Alberquilla, and Boris Grot. 2019. Stretch: Balancing qos and throughput for colocated server workloads on smt cores. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 15–27.
- [77] Daniel Marjamäki. 2013. Cppcheck: a tool for static c/c++ code analysis. URL: <https://cppcheck.sourceforge.io> (2013).
- [78] B. Maronga, S. Banzhaf, C. Burmeister, T. Esch, R. Forkel, D. Fröhlich, V. Fuka, K. F. Gehrke, J. Geletić, S. Giersch, T. Gronemeier, G. Groß, W. Heldens, A. Hellsten, F. Hoffmann, A. Inagaki, E. Kadasch, F. Kanani-Sühring, K. Ketelsen, B. A. Khan, C. Knigge, H. Knoop, P. Krč, M. Kurppa, H. Maamari, A. Matzarakis, M. Mauder, M. Pallasch, D. Pavlik, J. Pfafferoth, J. Resler, S. Rissmann, E. Russo, M. Salim, M. Schrempf, J. Schwenkel, G. Seckmeyer, S. Schubert, M. Sühring, R. von Tils, L. Vollmer, S. Ward, B. Witha, H. Wurps, J. Zeidler, and S. Raasch. 2020. Overview of the PALM model system 6.0. *Geoscientific Model Development* 13, 3 (2020), 1335–1372.
- [79] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. 2020. Mlperf training benchmark. *Proceedings of Machine Learning and Systems* 2 (2020), 336–349.
- [80] Simon McIntosh-Smith, Matthew Martineau, Tom Deakin, Grzegorz Pawelczak, Wayne Gaudin, Paul Garrett, Wei Liu, Richard Smedley-Stevenson, and David Beckingsale. 2017. Tealeaf: A mini-application to enable design-space explorations for iterative sparse linear solvers. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 842–849.
- [81] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. 2010. Introducing the graph 500. *Cray Users Group (CUG)* 19, 45-74 (2010), 22.
- [82] Seonjin Na, Geonhwa Jeong, Byung H Ahn, Aaron Jezghani, Jeffrey Young, Christopher J Hughes, Tushar Krishna, and Hyesoon Kim. 2025. Flexinfer: Flexible llm inference with cpu computations. *Proceedings of Machine Learning and Systems* 7 (2025).
- [83] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. 2024. Understanding performance implications of llm inference on cpus. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 169–180.
- [84] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 57–70.
- [85] Arash Nasr-Esfahany, Mohammad Alizadeh, Victor Lee, Hanna Alam, Brett W. Coon, David Culler, Vidushi Dadu, Martin Dixon, Henry M. Levy, Santosh Pandey, Parthasarathy Ranganathan, and Amir Yazdanbakhsh. 2025. Concorde: Fast and Accurate CPU Performance Modeling with Compositional Analytical-ML Fusion. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1480–1494.
- [86] Nevine Nassif, Ashley O. Munch, Carleton L. Molnar, Gerald Pasdast, Sitaraman V. Lyer, Zibing Yang, Oscar Mendoza, Mark Huddart, Srikrishnan Venkataraman, Sireesha Kandula, Rafi Marom, Alexandra M. Kern, Bill Bowhill, David R. Mulvihill, Srikanth Nimmagadda, Varma Kalidindi, Jonathan Krause, Mohammad M. Haq, Roopali Sharma, and Kevin Duda. 2022. Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 44–46. <https://doi.org/10.1109/ISSCC42614.2022.9731107>
- [87] Agustín Navarro-Torres, Jesús Alastruey-Benedé, Pablo Ibáñez-Marín, and Víctor Viñals-Yúfera. 2019. Memory hierarchy characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP. *Plos one* 14, 8 (2019), e0220135.
- [88] Nicholas Nethercote, Peter J Stuckey, Ralph Becket, Sebastian Brand, Gregory J Duck, and Guido Tack. 2007. MiniZinc: Towards a standard CP modelling language. In *International conference on principles and practice of constraint programming*. Springer, 529–543.
- [89] NVIDIA. 2026. NVIDIA Grace CPU Superchip Whitepaper. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-cpu-superchip>.
- [90] Biswabandan Panda. 2023. Clip: Load criticality based data prefetching for bandwidth-constrained many-core systems. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 714–727.
- [91] Reena Panda, Shuang Song, Joseph Dean, and Lizy K John. 2018. Wait of a decade: Did spec cpu 2017 broaden the performance horizon?. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 271–282.
- [92] Irma Esmer Papazian. 2020. New 3rd Gen Intel® Xeon® Scalable Processor (Codename: Ice Lake-SP).. In *Hot Chips Symposium*. 1–22.
- [93] Gyeongseo Park, Minh Kim, Ki-Dong Kang, Yunhyeong Jeon, Seulki Kim, and Daehoon Kim. 2025. EcoCore: Dynamic Core Management for Improving Energy Efficiency in Latency-Critical Applications. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. 1132–1146.
- [94] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [95] Tirthak Patel and Devesh Tiwari. 2020. Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 193–206.
- [96] Erez Perelman, Greg Hamerly, Michael Van Biesbrouck, Timothy Sherwood, and Brad Calder. 2003. Using simpoint for accurate and efficient simulation. *ACM SIGMETRICS Performance Evaluation Review* 31, 1 (2003), 318–319.
- [97] Aashish Phansalkar, Ajay Joshi, Lieven Eeckhout, and Lizy Kurian John. 2005. Measuring program similarity: Experiments with SPEC CPU benchmark suites. In *IEEE International Symposium on Performance Analysis of Systems and Software, 2005. ISPASS 2005*. IEEE, 10–20.
- [98] Aashish Phansalkar, Ajay Joshi, and Lizy K John. 2007. Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite. In *Proceedings of the 34th annual International Symposium on Computer architecture*. 412–423.
- [99] James C Phillips, Rosemary Braun, Wei Wang, James Gumbart, Emad Tajkhorshid, Elizabeth Villa, Christophe Chipot, Robert D Skeel, Laxmikant Kale, and Klaus Schulten. 2005. Scalable molecular dynamics with NAMD. *Journal of computational chemistry* 26, 16 (2005), 1781–1802.
- [100] Thomas Pohl, Markus Kowarschik, Jens Wilke, Klaus Iglberger, and Ulrich Rüde. 2003. Optimization and profiling of the cache performance of parallel lattice Boltzmann codes. *Parallel Processing Letters* 13, 04 (2003), 549–560.
- [101] CloverLeaf Mantevo Project. 2026. Compressible Euler equations on a Cartesian grid. <https://github.com/Mantevo/CloverLeaf>.
- [102] The Tukaani Project. 2026. XZ Utils. <https://github.com/tukaani-project/xz>.
- [103] Aleksandar Prokopec, Andrea Rosà, David Leopoldsdeder, Gilles Duboscq, Petr Tůma, Martin Studener, Lubomír Bulej, Yudi Zheng, Alex Villazón, Doug Simon, Thomas Würthinger, and Walter Binder. 2019. Renaissance: Benchmarking suite for parallel applications on the jvm. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 31–47.
- [104] Ritik Raj, Hong Wang, and Tushar Krishna. 2025. A CPU-Centric Perspective on Agentic AI. *arXiv preprint arXiv:2511.00739* (2025).
- [105] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.
- [106] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Likhomotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. Mlperf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.
- [107] Vijay Janapa Reddi, David Kanter, Peter Mattson, Jared Duke, Thai Nguyen, Ramesh Chukka, Ken Shiring, Koan-Sin Tan, Mark Charlebois, William Chou, Mostafa El-Khany, Jungwook Hong, Tom St. John, Cindy Trinh, Michael Buch, Mark Mazumder, Relia Markovic, Thomas Atta, Fatih Cakir, Masoud Charkhabi, Xiaodong Chen, Cheng-Ming Chiang, Dave Dexter, Terry Heo, Gunther Schmuelling, Maryam Shabani, and Dylan Zika. 2022. MLPerf mobile inference benchmark: An industry-standard open-source machine learning benchmark for on-device AI. *Proceedings of Machine Learning and Systems* 4 (2022), 352–369.
- [108] George F Riley and Thomas R Henderson. 2010. The ns-3 network simulator. In *Modeling and tools for network simulation*. Springer, 15–34.
- [109] Tord Romstad, Marco Costalba, Joona Kiiski, and G Linscott. 2008. Stockfish: A strong open source chess engine. <https://stockfishchess.org>.
- [110] Norbert Schmitt, James Bucek, John Beckett, Aaron Cragin, Klaus-Dieter Lange, and Samuel Kounev. 2020. Performance, power, and energy-efficiency impact analysis of compiler optimizations on the spec cpu 2017 benchmark suite. In

2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC). IEEE, 292–301.

- [111] Norbert Schmitt, James Bucek, Klaus-Dieter Lange, and Samuel Kounev. 2020. Energy efficiency analysis of compiler optimizations on the spec cpu 2017 benchmark suite. In *Companion of the ACM/SPEC International Conference on Performance Engineering*. 38–41.
- [112] Yalong Shan, Yongkui Yang, Xuehai Qian, and Zhibin Yu. 2024. Guser: A GPGPU power stressmark generator. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1111–1124.
- [113] Alexander F Shchepetkin and James C McWilliams. 2005. The regional oceanic modeling system (ROMS): a split-explicit, free-surface, topography-following-coordinate oceanic model. *Ocean modelling* 9, 4 (2005), 347–404.
- [114] Yongming Shen, Michael Ferdman, and Peter Milder. 2017. Maximizing CNN accelerator efficiency through resource partitioning. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 535–547.
- [115] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. 2002. Automatically characterizing large scale program behavior. *ACM SIGPLAN Notices* 37, 10 (2002), 45–57.
- [116] Sarabjeet Singh and Manu Awasthi. 2019. Memory centric characterization and analysis of spec cpu2017 suite. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*. 285–292.
- [117] Teja Singh, Spence Oliver, Sundar Rangarajan, Shane Southard, Carson Henrion, Alex Schaefer, Brett Johnson, Sarah Bartaszewicz Tower, Kathy Hoover, Deepesh John, Ted Antoniadis, Shravan Lakshman, Vibhor Mittal, Brian Kasprzyk, Ross McCoy, Kurt Mohlman, Anitha Mohan, Hon-Hin Wong, Daryl Lieu, Russell Schreiber, Sahilpreet Singh, Nick Lance, Darryl Prudich, Justin Coppin, Tim Jackson, Anita Karegar, Ryan Miller, Sabeesh Balagangadharan, James Pistole, Wilson Li, and Michael McCabe. 2025. “Zen 5”: The AMD High-Performance 4nm x86-64 Microprocessor Core. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 68. 1–3. <https://doi.org/10.1109/ISSCC49661.2025.10904529>
- [118] Alan Smith, Gabriel H. Loh, Michael J. Schulte, Mike Ignatowski, Samuel Nafziger, Mike Mantor, Mark Fowler Nathan Kalyanasundharam, Vamsi Alla, Nicholas Malaya, Joseph L. Greathouse, Eric Chapman, and Raja Swaminathan. 2024. Realizing the AMD Exascale Heterogeneous Processor Vision: Industry Product. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 876–889.
- [119] Don Soltis and Stephen Robinson. 2023. The Next Generation of High Performance, Energy-Efficient Computing: Intel® Xeon® Processors Built on Efficient-Core.. In *HCS*. 1–16.
- [120] Kevin Song, Jiacheng Yang, Zixuan Wang, Jishen Zhao, Sihang Liu, and Gennady Pekhimenko. 2025. Hybridtier: an adaptive and lightweight cxl-memory tiering system. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 112–128.
- [121] Cloyce D Spradling. 2007. SPEC CPU2006 benchmark tools. *ACM SIGARCH Computer Architecture News* 35, 1 (2007), 130–134.
- [122] Ranjan Hebbar SR, Mounika Ponugoti, and Aleksandar Milenković. 2019. Battle of compilers: An experimental evaluation using spec cpu2017. In *2019 South-eastCon*. IEEE, 1–8.
- [123] Akshitha Sriraman, Abhishek Dhanotia, and Thomas F Wenisch. 2019. Softsku: Optimizing server architectures for microservice diversity@ scale. In *Proceedings of the 46th International Symposium on Computer Architecture*. 513–526.
- [124] Richard M Stallman. 2002. GNU compiler collection internals. *Free Software Foundation* 46 (2002), 765.
- [125] Jovan Stojkovic, Abraham Farrell, Gerasimos Gerogiannis, Zhangxiaowen Gong, C Hughes, and Josep Torrellas. 2026. Dorado: Clustered Hardware Cache Coherence for 1,000+ Cores. In *Proceedings of the 53rd Annual International Symposium on Computer Architecture (ISCA’26)*.
- [126] Jovan Stojkovic, Abraham Farrell, Zhangxiaowen Gong, Christopher J Hughes, and Josep Torrellas. 2026. AccelFlow: Orchestrating an On-Package Ensemble of Fine-Grained Accelerators for Microservices. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–17.
- [127] Jovan Stojkovic, Chunao Liu, Muhammad Shahbaz, and Josep Torrellas. 2023. μ Manycore: A Cloud-Native CPU for Tail at Scale. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [128] Wei Su, Abhishek Dhanotia, Carlos Torres, Jayneel Gandhi, Neha Gholkar, Shobhit Kanauria, Maxim Naumov, Kalyan Subramanian, Valentin Andrei, Yifan Yuan, and Chunqiang Tang. 2025. DCPerf: An Open-Source, Battle-Tested Performance Benchmark Suite for Datacenter Workloads. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1717–1730.
- [129] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
- [130] John R Tramm, Andrew R Siegel, Benoit Forget, and Colin Josey. 2014. Performance analysis of a reduced data movement algorithm for neutron cross section data in monte carlo simulations. In *International Conference on Exascale Applications and Software*. Springer, 39–56.
- [131] Arya Tschand, Arun Tejusve Raghunath Rajan, Sachin Idgunji, Anirban Ghosh, Jeremy Holleman, Csaba Kiraly, Pawan Ambalkar, Ritika Borkar, Ramesh Chukka, Trevor Cockrell, Oliver Curtis, Grigori Fursin, Miro Hodak, Hiwot Kassa, Anton Lokhmotov, Dejan Miskovic, Yuechao Pan, Manu Prasad Manmathan, Liz Raymond, Tom St. John, Arjun Suresh, Rowan Taubitz, Sean Zhan, Scott Wasson, David Kanter, and Vijay Janapa Reddi. 2025. MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μ Watts to MWatts for Sustainable AI. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1201–1216.
- [132] Taegeon Um, Byungsoo Oh, Byeongchan Seo, Minhyeok Kweun, Goeun Kim, and Woo-Yeon Lee. 2023. Fastflow: Accelerating deep learning model training with smart offloading of input data pipeline. *Proceedings of the VLDB Endowment* 16, 5 (2023), 1086–1099.
- [133] Guido Van Rossum. 1995. *Python reference manual*. Vol. 111. Centrum voor Wiskunde en Informatica Amsterdam.
- [134] András Varga and Rudolf Hornig. 2008. An overview of the OMNeT++ simulation environment. In *Proceedings of the 1st international conference on Simulation tools and techniques for communications, networks and systems & workshops*. 1–10.
- [135] Markus Velten, Robert Schöne, Thomas Ilsche, and Daniel Hackenberg. 2022. Memory performance of AMD EPYC Rome and Intel Cascade Lake SP server processors. In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering*. 165–175.
- [136] Doug Walker, Michael Dolan, and Patrick Hodoul. 2020. The ASWF takes OpenColorIO to the next level. In *Proceedings of the 2020 Digital Production Symposium*. 1–10.
- [137] Siqi Wang, Tianyu Feng, Hailong Yang, Xin You, Bangduo Chen, Tongxuan Liu, Zhongzhi Luan, and Depei Qian. 2024. AtRec: Accelerating recommendation model training on CPUs. *IEEE Transactions on Parallel and Distributed Systems* 35, 6 (2024), 905–918.
- [138] Chris Welty. 2026. NTest othello program. <https://github.com/weltyc/ntest>.
- [139] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxin Li, Jingxiang Hou, Xu Dai, Chao Li, Shaolin Wei, Yang Hu, and Shouyi Yin. 2025. WSC-LLM: Efficient LLM Service and Architecture Co-exploration for Wafer-scale Chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1–17.
- [140] Jawad Haj Yahya, Haris Volos, Davide B. Bartolini, Georgia Antoniou, Jeremie S. Kim, Zhe Wang, Kleovoulos Kalaitzidis, Tom Rollet, Zhirui Chen, Ye Geng, Onur Mutlu, and Yiannakis Sazeides. 2022. Agilewatts: An energy-efficient cpu core idle-state architecture for latency-sensitive server applications. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 835–850.
- [141] Dong Ye, Joydeep Ray, Christophe Harle, and David Kaeli. 2006. Performance characterization of SPEC CPU2006 integer benchmarks on x86-64 architecture. In *2006 IEEE International Symposium on Workload Characterization*. IEEE, 120–127.
- [142] Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. 2024. Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference. In *Proceedings of the 17th ACM International Systems and Storage Conference*. 91–103.
- [143] Mohamed Zahran. 2019. *Heterogeneous computing: Hardware and software perspectives*. Morgan & Claypool.
- [144] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 161–170.
- [145] Shiqing Zhang, Mahmood Naderan-Tahan, Magnus Jahre, and Lieven Eeckhout. 2023. Characterizing multi-chip GPU data sharing. *ACM Transactions on Architecture and Code Optimization* 20, 4 (2023), 1–24.
- [146] Yuxuan Zhang, Tanvir Ahmed Khan, Gilles Pokam, Baris Kasikci, Heiner Litz, and Joseph Devietti. 2022. Ocolos: Online code layout optimizations. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 530–545.
- [147] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jishi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shiron Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y.X. Wei. 2025. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1731–1745.
- [148] Kaiyang Zhao, Kaiwen Xue, Ziqi Wang, Dan Schatzberg, Leon Yang, Antonis Manousis, Johannes Weiner, Rik Van Riel, Bikash Sharma, Chunqiang Tang, and Dimitrios Skarlatos. 2023. Contiguities: The pursuit of physical memory contiguity in datacenters. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [149] Laiping Zhao, Yanan Yang, Yiming Li, Xian Zhou, and Keqiu Li. 2021. Understanding, predicting and scheduling serverless workloads under partial interference. In *Proceedings of the International conference for high performance computing, networking, storage and analysis*. 1–15.

- [150] Mark Zhao, Niket Agarwal, Aarti Basant, Buğra Gedik, Satadru Pan, Mustafa Ozdal, Rakesh Komuravelli, Jerry Pan, Tianshu Bao, Haowei Lu, Sundaram Narayanan, Jack Langman, Kevin Wilfong, Harsha Rastogi, Carole-Jean Wu, Christos Kozyrakis, and Parik Pol. 2022. Understanding data storage and ingestion for large-scale deep recommendation model training: Industrial product. In *Proceedings of the 49th annual international symposium on computer architecture*. 1042–1057.
- [151] Yufeng Zhou, Alan L Cox, Sandhya Dwarkadas, and Xiaowan Dong. 2023. The Impact of Page Size and Microarchitecture on Instruction Address Translation Overhead. *ACM Transactions on Architecture and Code Optimization* 20, 3 (2023), 1–25.
- [152] Zhuangzhuang Zhou, Vaibhav Gogte, Nilay Vaish, Chris Kennelly, Patrick Xia, Svilen Kanev, Tipp Moseley, Christina Delimitrou, and Parthasarathy Ranganathan. 2024. Characterizing a memory allocator at warehouse scale. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 192–206.

A Appendix

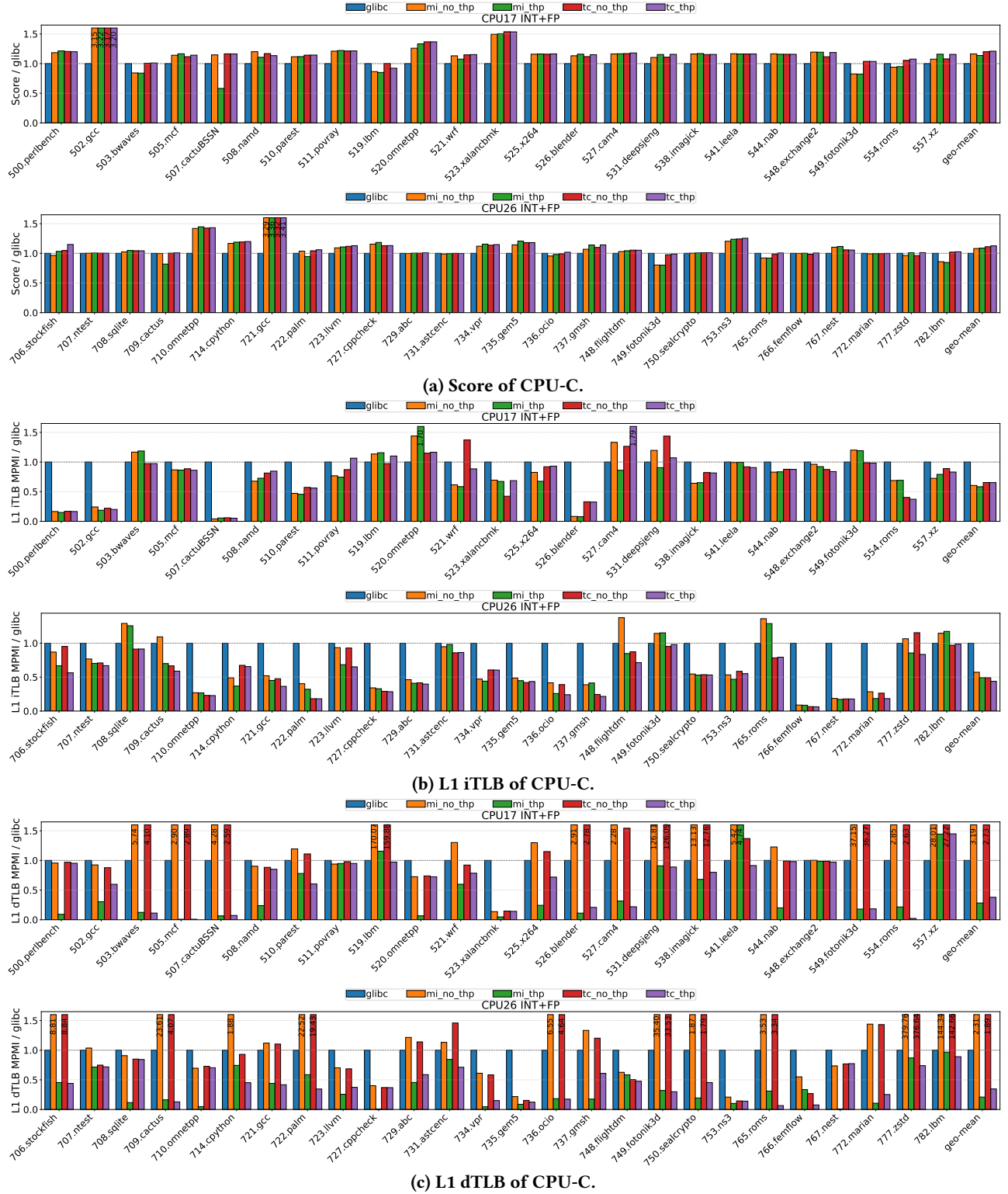


Figure 17: Detailed per-workload performance of different allocators on SPEC CPU17 and SPEC CPU26 (Figure 6, CPU-C).

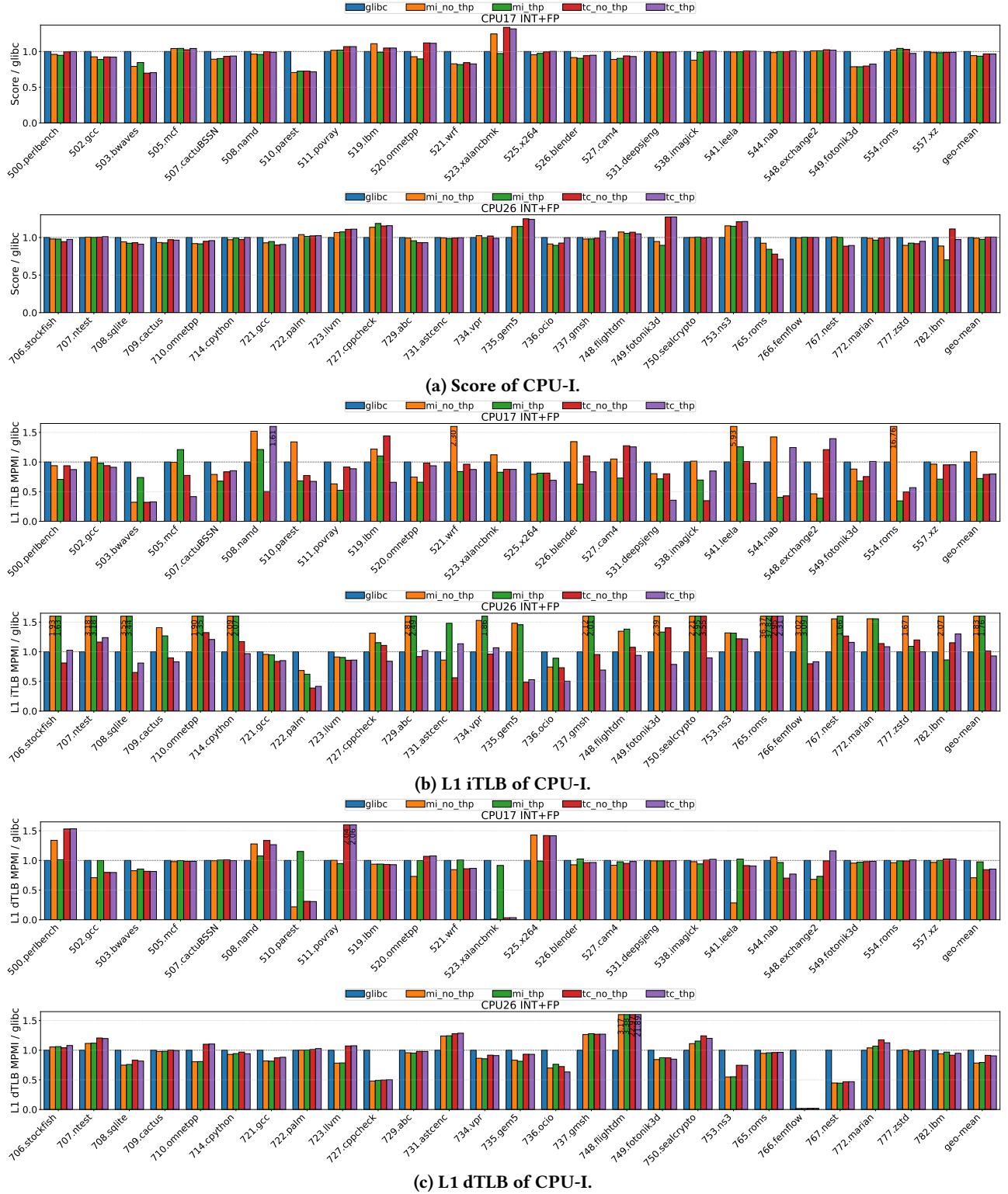


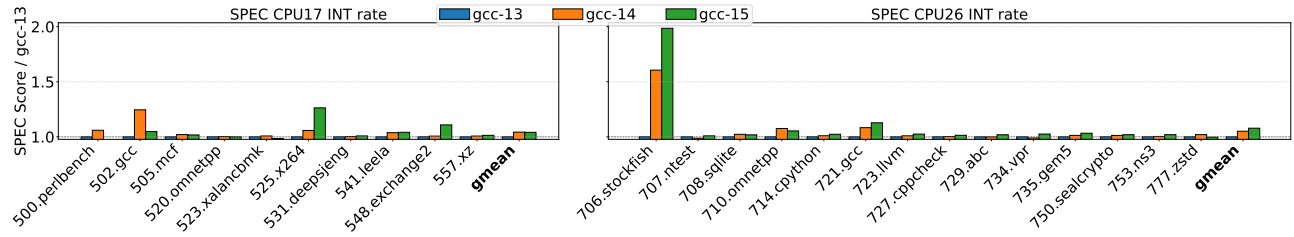
Figure 18: Detailed per-workload performance of different allocators on *SPEC CPU17* and *SPEC CPU26* (Figure 6, CPU-I).

Table 8: Detailed per-workload RSS of SPEC CPU17 (appendix for Figure 7a), in GB.

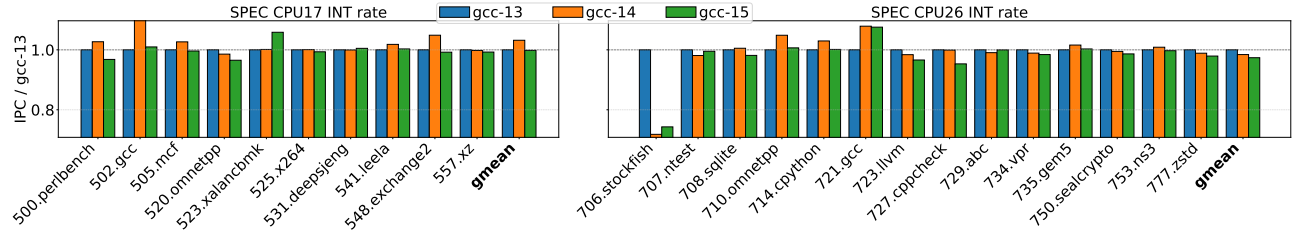
Workload	C_glibc	C_mi_no_thp	C_mi_thp	C_tc_no_thp	C_tc_thp	I_glibc	I_mi_no_thp	I_mi_thp	I_tc_no_thp	I_tc_thp
500.perlbench_r	0.54	0.57	0.58	0.57	0.57	0.56	0.59	0.59	0.60	0.59
502.gcc_r	1.66	1.63	1.64	1.68	1.67	1.67	1.66	1.64	1.70	1.69
503.bwaves_r	1.15	1.15	1.18	1.16	1.16	1.16	1.16	1.16	1.17	1.17
505.mcf_r	0.94	0.94	0.95	1.23	1.23	0.96	0.96	0.96	1.24	1.24
507.cactuBSSN_r	1.12	1.12	1.20	1.12	1.12	1.14	1.14	1.14	1.15	1.15
508.namd_r	0.51	0.50	0.56	0.51	0.51	0.53	0.54	0.54	0.55	0.55
510.parest_r	0.75	0.77	0.80	0.81	0.81	0.77	0.81	0.81	0.84	0.85
511.povray_r	0.45	0.44	0.45	0.45	0.45	0.42	0.43	0.46	0.46	0.48
519.lbm_r	0.75	0.75	0.75	0.75	0.75	0.76	0.76	0.76	0.77	0.77
520.omnetpp_r	0.58	0.58	0.58	0.58	0.59	0.60	0.59	0.59	0.60	0.60
521.wrf_r	0.54	0.55	0.61	0.54	0.55	0.56	0.58	0.58	0.58	0.58
523.xalancbmk_r	0.81	0.81	0.82	0.84	0.84	0.83	0.83	0.85	0.86	0.86
525.x264_r	0.50	0.50	0.56	0.51	0.51	0.52	0.53	0.53	0.53	0.53
526.blender_r	0.96	1.08	1.10	1.01	1.01	0.98	1.12	1.12	1.05	1.05
527.cam4_r	1.22	1.24	1.31	1.25	1.25	1.23	1.26	1.26	1.27	1.27
531.deepsjeng_r	1.03	1.03	1.03	1.04	1.04	1.04	1.04	1.04	1.05	1.05
538.imagick_r	0.63	0.72	0.72	0.72	0.72	0.64	0.73	0.73	0.74	0.74
541.leela_r	0.40	0.40	0.45	0.45	0.40	0.41	0.42	0.47	0.42	0.42
544.nab_r	0.49	0.53	0.54	0.51	0.52	0.51	0.55	0.55	0.52	0.52
548.exchange2_r	0.40	0.46	0.46	0.45	0.44	0.47	0.41	0.47	0.42	0.46
549.fotonik3d_r	1.17	1.18	1.20	1.18	1.18	1.19	1.19	1.19	1.20	1.20
554.roms_r	1.17	1.17	1.23	1.18	1.18	1.18	1.19	1.19	1.20	1.20
557.xz_r	1.11	1.16	1.17	1.10	1.28	1.11	1.16	1.16	1.20	1.29

Table 9: Detailed per-workload RSS of SPEC CPU26 (appendix for Figure 7a), in GB.

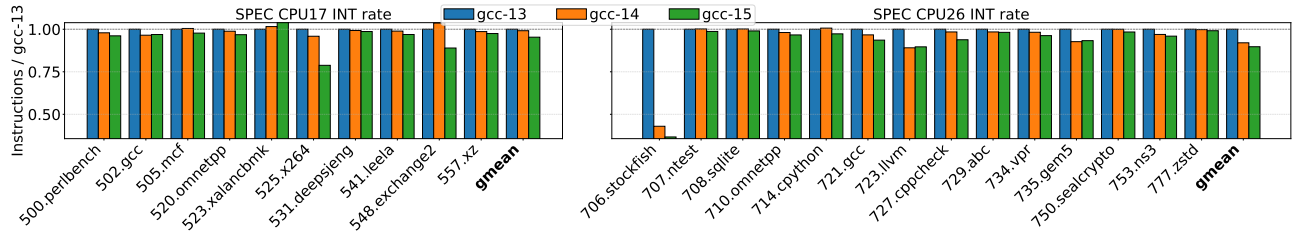
Workload	C_glibc	C_mi_no_thp	C_mi_thp	C_tc_no_thp	C_tc_thp	I_glibc	I_mi_no_thp	I_mi_thp	I_tc_no_thp	I_tc_thp
706.stockfish_r	2.06	2.06	2.14	2.13	2.13	2.08	2.15	2.15	2.15	2.15
707.ntest_r	0.64	0.63	0.64	0.64	0.64	0.66	0.67	0.67	0.67	0.66
708.sqlite_r	1.59	1.59	1.43	1.35	1.35	1.61	1.43	1.45	1.35	1.35
709.cactus_r	2.05	2.05	2.17	2.05	2.05	2.08	2.09	2.09	2.09	2.09
710.omnetpp_r	1.08	1.15	1.07	1.13	1.09	1.16	1.14	1.15	1.10	1.08
714.cpython_r	1.53	1.51	1.57	1.69	1.71	1.53	1.57	1.58	1.69	1.70
721.gcc_r	1.34	1.34	1.33	1.72	1.72	1.38	1.36	1.35	1.76	1.75
722.palm_r	2.13	2.15	2.19	2.18	2.19	2.14	2.19	2.19	2.21	2.21
723.llvm_r	1.76	1.76	1.69	1.72	1.72	1.79	1.70	1.70	1.75	1.75
727.cppcheck_r	0.97	0.97	0.95	0.96	0.96	0.99	0.97	0.97	0.98	0.98
729.abc_r	1.84	1.85	1.86	1.86	1.87	1.87	1.86	1.86	1.89	1.89
731.astcenc_r	0.65	0.64	0.64	0.64	0.64	0.67	0.67	0.67	0.67	0.67
734.vpr_r	1.98	2.00	2.06	1.80	1.81	2.03	2.02	2.12	1.85	1.85
735.gem5_r	1.13	1.10	1.06	1.05	1.10	1.16	1.07	1.07	1.08	1.08
736.ocio_r	1.92	1.92	1.93	1.92	1.93	1.94	1.94	1.94	1.95	1.95
737.gmsh_r	1.39	1.39	1.47	1.45	1.45	1.42	1.49	1.56	1.52	1.52
748.flightdm_r	0.64	0.65	0.65	0.64	0.64	0.67	0.67	0.67	0.67	0.67
749.fotonik3d_r	2.11	2.11	2.15	2.12	2.12	2.13	2.14	2.14	2.15	2.15
750.sealcrypto_r	0.93	0.93	0.96	0.93	0.95	0.96	0.96	0.96	0.96	0.96
753.ns3_r	0.64	0.64	0.64	0.64	0.64	0.67	0.67	0.67	0.67	0.66
760.rocksdb_r	0.77	0.77	0.75	0.76	0.76	0.79	0.77	0.77	0.78	0.78
765.roms_r	1.56	1.56	1.62	1.57	1.58	1.59	1.60	1.60	1.62	1.62
766.femflow_r	0.73	0.71	0.69	0.71	0.72	0.75	0.72	0.72	0.75	0.75
767.nest_r	1.46	1.46	1.72	1.48	1.48	1.49	1.74	1.74	1.50	1.50
772.marian_r	1.44	1.44	1.42	1.39	1.39	1.46	1.46	1.46	1.39	1.39
777.zstd_r	0.84	0.84	0.84	0.84	0.84	0.86	0.86	0.86	0.86	0.86
782.lbm_r	2.03	2.03	2.03	2.03	2.04	2.05	2.05	2.05	2.06	2.06



(a) Score of INT Rate workloads.

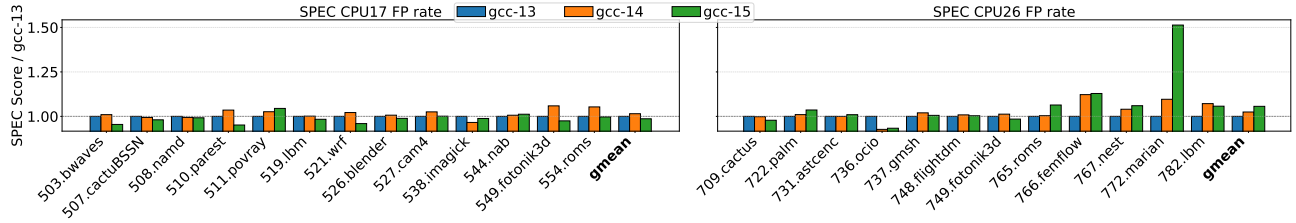


(b) IPC of INT Rate workloads.

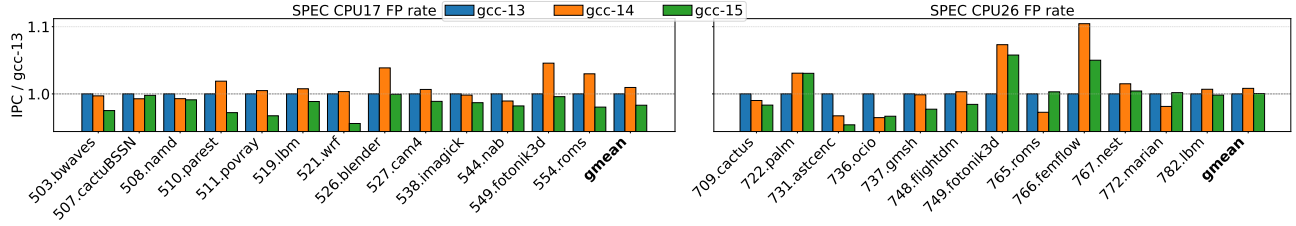


(c) Instruction counts of INT Rate workloads.

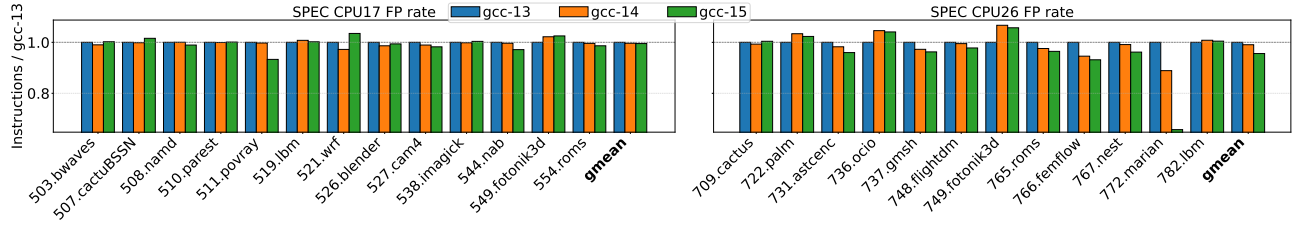
Figure 19: Detailed per-workload performance of different compilers on *SPEC CPU17* and *SPEC CPU26* INT Rate workloads (Figure 9).



(a) Score of FP Rate workloads.



(b) IPC of FP Rate workloads.



(c) Instruction counts of FP Rate workloads.

Figure 20: Detailed per-workload performance of different compilers on *SPEC CPU17* and *SPEC CPU26* FP Rate workloads (Figure 9).

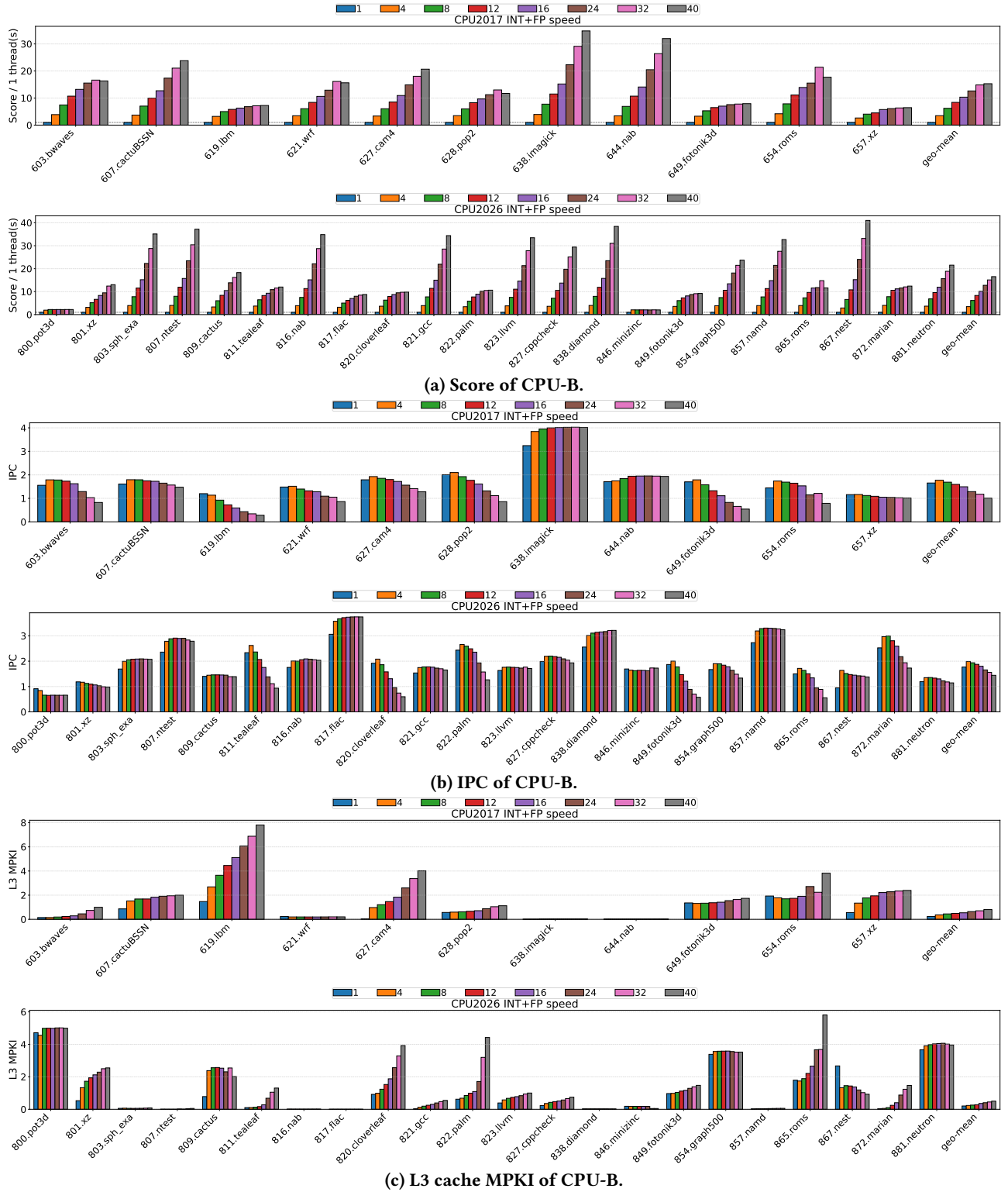


Figure 21: Detailed per-workload performance of Speed workload scaling on *SPEC CPU17* and *SPEC CPU26* (Figure 12, CPU-B).

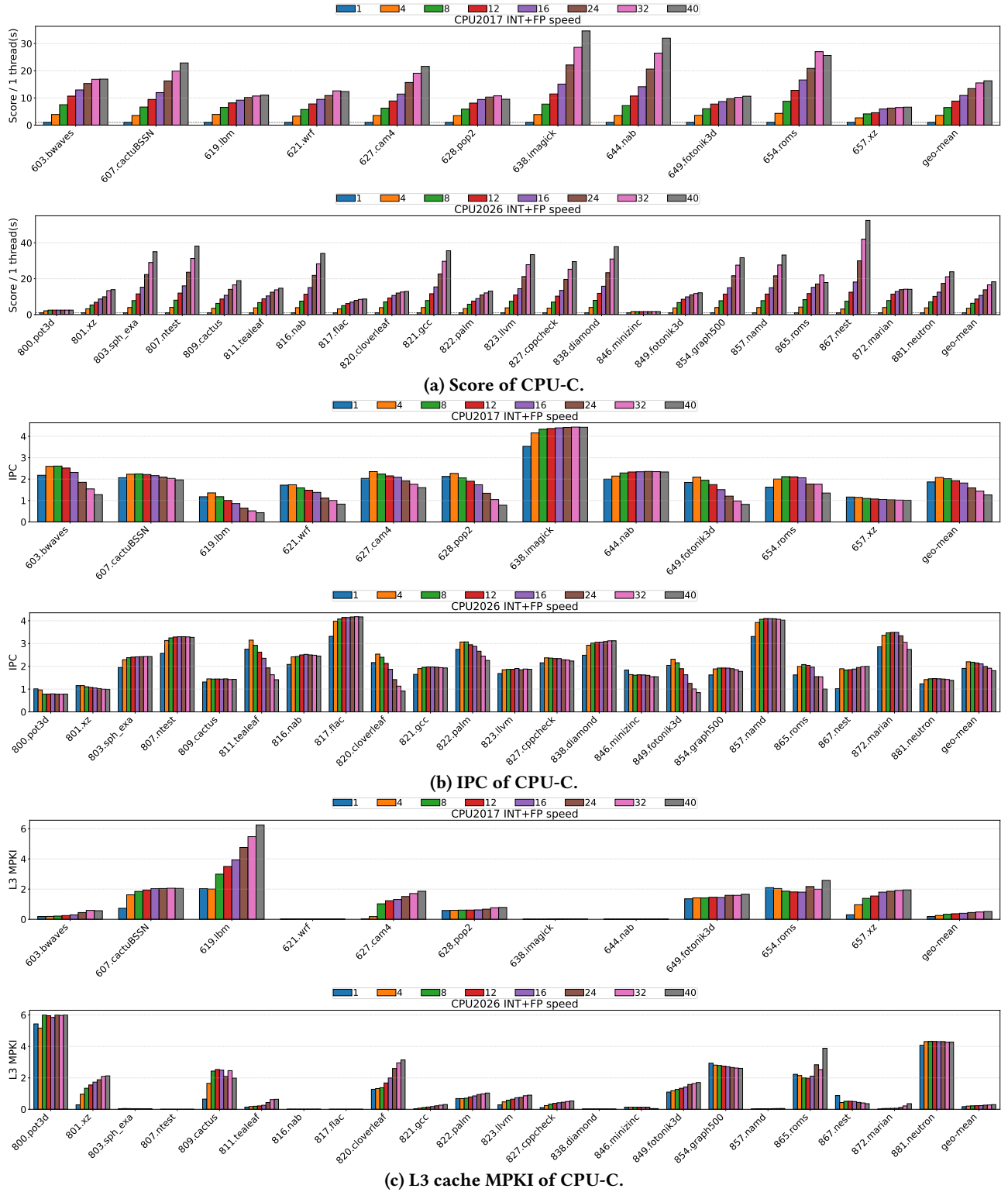


Figure 22: Detailed per-workload performance of Speed workload scaling on *SPEC CPU17* and *SPEC CPU26* (Figure 12, CPU-C).